Attention and transformers: alignment, self-attention and a first look inside

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

An encoder–decoder recurrent network (Note 14, Section 7) must squeeze a whole input sequence into one vector before it starts to produce the output. *Attention* removes this bottleneck: at every output step the decoder computes a fresh weighted average of all encoder states, with weights that measure how relevant each input position is right now. This note derives soft attention and its gradient, follows location-based windows, connectionist temporal classification and hard attention trained with REINFORCE, and arrives at *self-attention*, where a sequence attends to itself and recurrence disappears. The transformer is assembled piece by piece, with its costs, followed by decoder-only language models, the key–value cache and IO-aware attention. The last section opens the box: the residual stream, what attention patterns do and do not tell, ablation, activation patching and probes — the tools of the course project.

1 Why attention: the fixed-vector bottleneck

Attention is a learned, input-dependent weighting: the model computes a probability distribution over the parts of its input and reads mostly from where that distribution puts its mass. A translation model weights the source words while it writes each target word (Sections 2 and 3); an image model weights regions of an image, and may look at only one of them (Section 4); in a transformer every position weights all the others, and this *self-attention* replaces recurrence and convolution (Sections 5 and 6); a memory network weights the slots of a memory by how well they match a query. In every case the model chooses its own wiring diagram afresh for each input.

The sequence-to-sequence setting. The encoder–decoder of Note 14, Section 7, models a conditional distribution of an output sequence $\mathbf{y} = (y_1, \dots, y_{T'})$ given an input sequence $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$ of another length [27, 42, 8]. An encoder RNN reads $\mathbf{x}$, its final state $\mathbf{v}$ (called the context vector $\mathbf{s}$ in Note 14) is the only link to the decoder, and the decoder generates

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{i=1}^{T'} p(y_i \mid \mathbf{v}, y_1, \dots, y_{i-1}) \tag{1}$$

token by token. In this note *context vector* means instead the summary $\mathbf{c}_i$ that attention recomputes at every output step.

Two problems. Note 14, Section 11, lists the fixed-size bottleneck and the long paths among the limits of recurrence. Here we make both quantitative.

- *A bottleneck of fixed width.* However long the sentence, the decoder sees only the d numbers of $\mathbf{v}$. With $d = 1000$, a sentence of 50 words gets 20 numbers per word, a paragraph of 500

words only two.

- *Long paths.* The information about $\mathbf{x}_j$ reaches output step i only after passing through $T - j$ encoder steps and i decoder steps, and the gradient travels the same path backwards. Note 14, Section 4, showed that gradients through many recurrent steps vanish or explode; a path of length $T - j + i$ makes the first words of a long sentence the hardest to learn from.

Bahdanau, Cho and Bengio measured the effect directly: the translation quality of an RNN encoder–decoder fell steadily with the length of the source sentence, while the same model with attention kept its quality on long sentences [4].

> **Key idea** Instead of one summary $\mathbf{v}$, keep all encoder states $\mathbf{h}_1, \dots, \mathbf{h}_T$, and let the decoder compute at each step i a new summary $\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j$, with weights $\alpha_{ij} \geq 0$, $\sum_j \alpha_{ij} = 1$, that depend on what the decoder needs at that step. The memory grows with the input, and every input position is connected to every output step by a path of length two (encoder state → context → decoder state).

2 Soft content-based attention

2.1 Context vectors and alignment weights

Let $\mathbf{h}_1, \dots, \mathbf{h}_T \in \mathbb{R}^{d_h}$ be the encoder states and $\mathbf{s}_i \in \mathbb{R}^{d_s}$ the decoder state from which y_i is predicted (it has already read y_{i-1}). In the attention model of Bahdanau, Cho and Bengio [4] the decoder computes, at output step i,

$$e_{ij} = a(\mathbf{s}_{i-1}, \mathbf{h}_j), \qquad j = 1, \dots, T, \tag{2}$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T} \exp(e_{ik})}, \tag{3}$$

$$\mathbf{c}_i = \sum_{j=1}^{T} \alpha_{ij} \, \mathbf{h}_j, \tag{4}$$

and then uses the *context vector* (or *glimpse*) $\mathbf{c}_i$ in place of the fixed $\mathbf{v}$:

$$\mathbf{s}_i = f(\mathbf{s}_{i-1}, y_{i-1}, \mathbf{c}_i), \qquad p(y_i \mid y_{<i}, \mathbf{x}) = \operatorname{softmax}\big(o(\mathbf{s}_i, y_{i-1}, \mathbf{c}_i)\big),$$

with f a GRU or LSTM update (Note 14, Sections 5 and 6) and o a small output network. The *score* or *alignment model* a measures how well the input around position j matches what the decoder is about to produce; the softmax (3) (Note 9, Section 4.4) turns the scores into a probability distribution over input positions; and the context is the expectation of the encoder state under that distribution. Figure 1 draws one step.

Why a bidirectional encoder. The state $\mathbf{h}_j$ should describe the input *around* position j, not just the prefix $\mathbf{x}_1, \dots, \mathbf{x}_j$. Bahdanau et al. therefore concatenate the states of a forward and a backward RNN, $\mathbf{h}_j = [\overrightarrow{\mathbf{h}}_j; \overleftarrow{\mathbf{h}}_j]$ (Note 14, Section 9). With a forward RNN alone, $\mathbf{h}_j$ would know nothing about the words after j, and an early word whose translation depends on a later one could not be matched correctly. Plain word embeddings, with no recurrence at all, would be worse still: the score could then only compare the decoder state with a single word out of context.

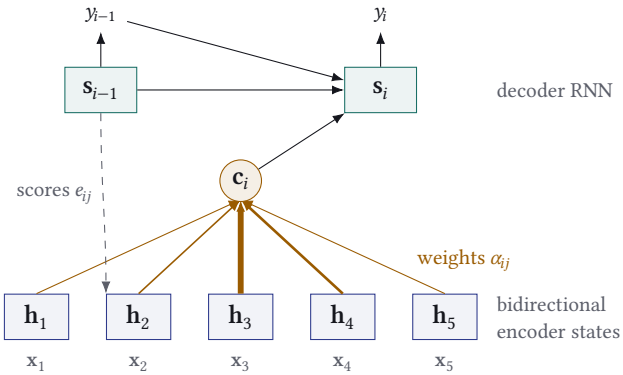


Figure 1. One decoder step with soft attention. The previous decoder state $\mathbf{s}_{i-1}$ is compared with every encoder state (dashed arrow); the softmax of the scores gives the weights α_{ij} (line widths); the weighted average $\mathbf{c}_i$ enters the update of the decoder state together with $\mathbf{s}_{i-1}$ and y_{i-1}.

Soft attention is a differentiable lookup. Everything in (2)–(4) is smooth, so the alignment is learned by backpropagation from translation pairs alone; nobody tells the model which words correspond.

2.2 Scoring functions

The original paper used a one-hidden-layer network for a, the *additive* (or concat) score. Luong, Pham and Manning compared simpler alternatives [28], and the transformer added a scaled version of the dot product. With $\mathbf{s}$ the decoder state and $\mathbf{h}$ an encoder state:

name	score $a(\mathbf{s},\mathbf{h})$	parameters	cost per pair
additive [4]	$\mathbf{w}_a^{\mathsf{T}} \tanh(W_a\mathbf{s} + U_a\mathbf{h})$	$d_a(d_s + d_h + 1)$	$O(d_a)$ after precomputing $U_a\mathbf{h}$
dot [28]	$\mathbf{s}^{\mathsf{T}}\mathbf{h}$	none ($d_s = d_h = d$)	$O(d)$
bilinear ("general") [28]	$\mathbf{s}^{\mathsf{T}} M \mathbf{h}$	$d_s d_h$	$O(d_h)$ after precomputing $M^{\mathsf{T}}\mathbf{s}$
scaled dot [43]	$\mathbf{s}^{\mathsf{T}}\mathbf{h}/\sqrt{d}$	none ($d_s = d_h = d$)	$O(d)$

Here $d = d_s = d_h$ wherever the two states must have the same width. The dot product needs no parameters but requires the decoder and encoder states to live in the same space; the bilinear form learns a map between the two spaces; the additive score is the most flexible and the slowest. Section 5.1 explains the factor $1/\sqrt{d}$.

Worked example — 1. Three scores on three encoder states

Let $T = 3$ and $d = 2$ with $\mathbf{h}_1 = (1, 0)$, $\mathbf{h}_2 = (0, 1)$, $\mathbf{h}_3 = (1, 1)$ and the decoder state $\mathbf{s} = (1, 2)$. *Dot product:* $e = (1, 2, 3)$. Then $e^{e_j} = (2.718, 7.389, 20.086)$ with sum 30.193, so

$$\boldsymbol{\alpha} = (0.090, 0.245, 0.665), \qquad \mathbf{c} = 0.090\,\mathbf{h}_1 + 0.245\,\mathbf{h}_2 + 0.665\,\mathbf{h}_3 = (0.755, 0.910).$$

Scaled dot product: $e/\sqrt{2} = (0.707, 1.414, 2.121)$ gives the flatter $\boldsymbol{\alpha} = (0.140, 0.284, 0.576)$ and $\mathbf{c} = (0.716, 0.860)$. *Bilinear* with $M = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, which swaps the two components: $e = (2, 1, 3)$ and $\boldsymbol{\alpha} = (0.245, 0.090, 0.665)$; the learned matrix changed which state ranks second. *Additive* with $W_a = U_a = I$ and $\mathbf{w}_a = (1, 1)$: the hidden vectors are $\tanh(\mathbf{s} + \mathbf{h}_j) = (0.964, 0.964)$, $(0.762, 0.995)$, $(0.964, 0.995)$, so $e = (1.928, 1.757, 1.959)$ and $\boldsymbol{\alpha} = (0.348, 0.293, 0.359)$. The tanh is close to saturation, the scores barely differ and the weights are almost uniform: the scale of the parameters matters as much as the form of the score.

2.3 The gradient through attention

Backpropagation (Note 12, Section 3) needs the derivatives of (3) and (4). Suppose the gradient $\bar{\mathbf{c}} = \partial \ell / \partial \mathbf{c}$ of the loss with respect to the context has arrived from above (we drop the output index i).

Through the weighted sum. Since $\mathbf{c} = \sum_j \alpha_j \mathbf{h}_j$,

$$\frac{\partial \ell}{\partial \alpha_j} = \bar{\mathbf{c}}^{\mathsf{T}} \mathbf{h}_j =: g_j, \qquad \left.\frac{\partial \ell}{\partial \mathbf{h}_j}\right|_{\text{via } \mathbf{c}} = \alpha_j \, \bar{\mathbf{c}}. \tag{5}$$

Each encoder state receives the incoming gradient scaled by its own weight: positions the decoder did not look at receive almost nothing through this path.

Through the softmax. With $\alpha_j = e^{e_j} / \sum_k e^{e_k}$, the derivative is $\partial \alpha_j / \partial e_m = \alpha_j (\delta_{jm} - \alpha_m)$, that is, the Jacobian is $\operatorname{diag}(\boldsymbol{\alpha}) - \boldsymbol{\alpha}\boldsymbol{\alpha}^{\mathsf{T}}$ (Note 12, Section 2). Hence

$$\frac{\partial \ell}{\partial e_m} = \sum_j g_j \, \alpha_j (\delta_{jm} - \alpha_m) = \alpha_m \big(g_m - \bar{g} \big), \qquad \bar{g} = \sum_j \alpha_j g_j. \tag{6}$$

The gradient of a score is the weight of that position times how much more its value $\mathbf{h}_m$ increases the loss than the current average $\bar{g}$ does; descent lowers that score when $g_m > \bar{g}$ and raises it when $g_m < \bar{g}$. Three consequences follow. The score gradients sum to zero, because adding a constant to all scores changes nothing. A position with $\alpha_m \approx 0$ gets almost no gradient, so a position the model ignores can be difficult to discover again. And if the softmax is saturated, $\boldsymbol{\alpha} \approx$ one-hot, then $\alpha_m (g_m - \bar{g}) \approx 0$ for every m: a saturated attention distribution stops learning, one reason, at initialisation, for the scaling of Section 5.1. Finally the score gradients flow into the parameters of a and, a second time, into $\mathbf{h}_j$ and $\mathbf{s}$.

Worked example — 2. Backward pass through Worked example 1

Take the dot-product weights $\boldsymbol{\alpha} = (0.090, 0.245, 0.665)$ and the loss $\ell = \mathbf{u}^{\mathsf{T}} \mathbf{c}$ with $\mathbf{u} = (1, -1)$, so $\bar{\mathbf{c}} = \mathbf{u}$. Then $g_j = \mathbf{u}^{\mathsf{T}} \mathbf{h}_j = (1, -1, 0)$ and $\bar{g} = 0.090 - 0.245 = -0.155$, and by (6)

$$\frac{\partial \ell}{\partial \mathbf{e}} = \big(0.090 \cdot 1.155, \; 0.245 \cdot (-0.845), \; 0.665 \cdot 0.155 \big) = (0.104, -0.207, 0.103),$$

which sums to zero. Gradient descent ($\mathbf{e} \leftarrow \mathbf{e} - \eta \, \partial \ell / \partial \mathbf{e}$) would lower the scores of $\mathbf{h}_1$ and $\mathbf{h}_3$ and raise that of $\mathbf{h}_2$, the only state whose value decreases the loss ($g_2 = -1 < \bar{g}$). Through (5) the encoder states receive $\alpha_j \mathbf{u}$: $(0.090, -0.090)$, $(0.245, -0.245)$ and $(0.665, -0.665)$. Central differences with step 10^{-6} (Note 12, Section 4) reproduce the three score gradients to six digits.

2.4 The algorithm and its cost

Algorithm 1 computes one decoder step with additive attention. The products $U_a \mathbf{h}_j$ do not depend on the decoder and are computed once per input sequence.

Running time — attention decoder (Algorithm 1)

Per input: $O(T d_h d_a)$ once for P. *Per output step*: $O(d_a d_s)$ for $W_a \mathbf{s}$, $O(T d_a)$ for the scores, $O(T)$ for the softmax and $O(T d_h)$ for the context, on top of the $O(d_s^2 + d_s d_h)$ recurrent update and the $O(d_s V)$ output softmax over a vocabulary of size V. *Per sequence pair*: $O\big(T T' (d_a + d_h)\big)$ for attention, quadratic in the lengths, but every step is a vector–matrix product that runs fast on a GPU. *Memory*: $O(T d_h)$ for the encoder states, and the $T \times T'$ weights kept for the backward pass. *Training* with teacher forcing (Note 14, Section 7) costs about three times the forward pass; *prediction* with beam width B costs B times a greedy decode.

Algorithm 1 One decoder step of an RNN encoder–decoder with additive attention

Require: encoder states $H = [\mathbf{h}_1, \dots, \mathbf{h}_T]^\top \in \mathbb{R}^{T \times d_h}$; precomputed $P = HU_a^\top \in \mathbb{R}^{T \times d_a}$; previous state $\mathbf{s}_{i-1}$ and token y_{i-1}; parameters $W_a, \mathbf{w}_a$, decoder update f and output map o

function AttentionStep($\mathbf{s}_{i-1}, y_{i-1}; H, P$)
  $\mathbf{r} \leftarrow W_a \mathbf{s}_{i-1}$ ▷ $O(d_a d_s)$
  $e_j \leftarrow \mathbf{w}_a^\top \tanh(\mathbf{r} + P_{j,:})$ for $j = 1, \dots, T$ ▷ add a vector to a matrix, $O(T d_a)$
  $e^* \leftarrow \max_j e_j$; $\alpha_j \leftarrow \exp(e_j - e^*) / \sum_k \exp(e_k - e^*)$ ▷ stable softmax
  $\mathbf{c}_i \leftarrow H^\top \boldsymbol{\alpha}$ ▷ vector–matrix product, $O(T d_h)$
  $\mathbf{s}_i \leftarrow f(\mathbf{s}_{i-1}, y_{i-1}, \mathbf{c}_i)$ ▷ GRU or LSTM update
  return $\mathbf{s}_i$, $p(y_i \mid \cdot) = \mathrm{softmax}\big(o(\mathbf{s}_i, y_{i-1}, \mathbf{c}_i)\big)$, and $\boldsymbol{\alpha}$ for inspection
end function

2.5 Reading alignments

The matrix of weights α_{ij}, with output steps as rows and input positions as columns, can be drawn as an image. Between languages with a similar word order it is close to diagonal. Figure 2 shows a pair with a different order: English "I read the red book" and Turkish "kırmızı kitabı okudum". The verb moves to the end, and the subject "I" and the article "the" have no word of their own: they become the personal ending of "okudum" and the accusative ending of "kitabı", which marks a definite object. A row with one dominant entry is a one-to-one correspondence. A row with two large entries is a Turkish word that does the work of two English words, and the order of the large entries from row to row shows the reordering. Section 7.2 will qualify how much such pictures explain.

	I	read	the	red	book
kırmızı	0.04	0.05	0.05	0.76	0.10
kitabı	0.04	0.04	0.22	0.07	0.63
okudum	0.25	0.56	0.05	0.06	0.08

Figure 2. Illustrative attention weights for translating "I read the red book" into Turkish (rows: output words; columns: input words; darker means larger). The weights are softmaxes of hand-chosen scores, not the output of a trained model.

Strengths and weaknesses. The length experiment of Bahdanau et al. quoted in Section 1 is the main argument for soft attention. Its weakness shows when input and output are aligned *monotonically*, as speech frames and the letters of the transcript: content alone is then a weak guide (two occurrences of the same sound look alike, and the model can jump back or skip), and the cost $O(TT')$ is a burden for inputs of thousands of steps. Sections 3 and 6.3 return to these problems.

> **In physics — attention weights are Boltzmann weights**
>
> Note 11, Section 2 (physics box), read the softmax as a Boltzmann distribution with energies $E_j = -e_j$, and Note 5, Section 3.3, read the factor $\sqrt{d_k}$ of Section 5.1 as a temperature. With $\alpha_j = e^{e_j/\tau}/Z$, the context vector is the thermal average $\langle \mathbf{h} \rangle$, and the free energy $F = -\tau \log Z$ has $-\partial F/\partial e_m = \alpha_m$. New here is the gradient (6) read as a fluctuation formula: $\partial \langle g \rangle / \partial e_m = \alpha_m (g_m - \langle g \rangle)/\tau$, which reduces to (6) at $\tau = 1$, is the covariance of g with the indicator of position m, divided by τ, as derivatives of thermal averages with respect to couplings are connected correlations. A frozen system has no fluctuations, hence no gradient — the statistical-mechanics reading of a saturated softmax.

3 Location-based attention and monotonic alignment

In content-based attention the weights depend on the *content* at each input position. In *location-based* attention they depend on *where* the model attended before: the new weights are computed relative to the old ones. The idea is older than content-based attention; it was introduced by Graves for handwriting synthesis [19], where a network must write a given text letter by letter and must move through the text in order.

3.1 A Gaussian window that only moves forward

Let $\boldsymbol{\chi}_1, \dots, \boldsymbol{\chi}_U$ be the one-hot vectors of the U characters to be written. (We follow Graves's notation: in this subsection $\mathbf{h}_t$ is the state of the network that writes, and α, β, κ are window parameters, not attention weights.) At each pen step t the network's hidden state $\mathbf{h}_t$ emits, through a linear layer, $3K$ numbers $(\hat{\alpha}_t, \hat{\beta}_t, \hat{\kappa}_t) \in \mathbb{R}^{3K}$ and defines a mixture of K Gaussian bumps over character positions u:

$$\phi(t,u) = \sum_{k=1}^{K} \alpha_t^k \exp\!\big(-\beta_t^k (\kappa_t^k - u)^2\big), \qquad \mathbf{w}_t = \sum_{u=1}^{U} \phi(t,u)\, \boldsymbol{\chi}_u, \tag{7}$$

with

$$\alpha_t^k = \exp(\hat{\alpha}_t^k), \qquad \beta_t^k = \exp(\hat{\beta}_t^k), \qquad \kappa_t^k = \kappa_{t-1}^k + \exp(\hat{\kappa}_t^k). \tag{8}$$

The *window vector* $\mathbf{w}_t$, a soft bag of the characters under the window, is passed to the other layers of the network and, at the next step, back to the layer that computes the window. The amplitude α sets the importance of a bump, β its sharpness (width about $1/\sqrt{\beta}$), and κ its centre. Because the centre advances by the positive amount $\exp(\hat{\kappa}_t^k)$, the window can only slide forwards: the alignment is monotonic by construction. The weights are not normalised; the network can switch the window off by making all amplitudes small.

Worked example — 3. A window sliding over five characters

Take $K = 1$, $\alpha = 1$, $\beta = 2$, $\kappa_0 = 0$, and offsets $\exp(\hat{\kappa}_t) = 1.0, 0.6, 0.6, 1.2$ for $t = 1, \dots, 4$, so that $\kappa_t = 1.0, 1.6, 2.2, 3.4$. For the positions $u = 1, \dots, 5$, $\phi(t,u) = \exp(-2(\kappa_t - u)^2)$ gives

t (κ_t)	$u=1$	$u=2$	$u=3$	$u=4$	$u=5$
1 (1.0)	1.000	0.135	0.000	0.000	0.000
2 (1.6)	0.487	0.726	0.020	0.000	0.000
3 (2.2)	0.056	0.923	0.278	0.002	0.000
4 (3.4)	0.000	0.020	0.726	0.487	0.006

For example $\phi(2,1) = e^{-2\cdot 0.36} = e^{-0.72} = 0.487$. If the five characters are all different, $\mathbf{w}_t$ simply lists these weights, one per letter; a repeated letter would collect the sum of its weights. The writer dwells on characters 1 and 2 for three steps (a letter takes several pen strokes), then moves on; the large offset at $t = 4$ skips ahead by more than a character. The window never moves back.

Figure 3 follows a window with two components over forty-six steps of a thirty-character text, with random offsets of mean about 0.6 characters per step: the bright band climbs monotonically, faster when the offsets are larger.

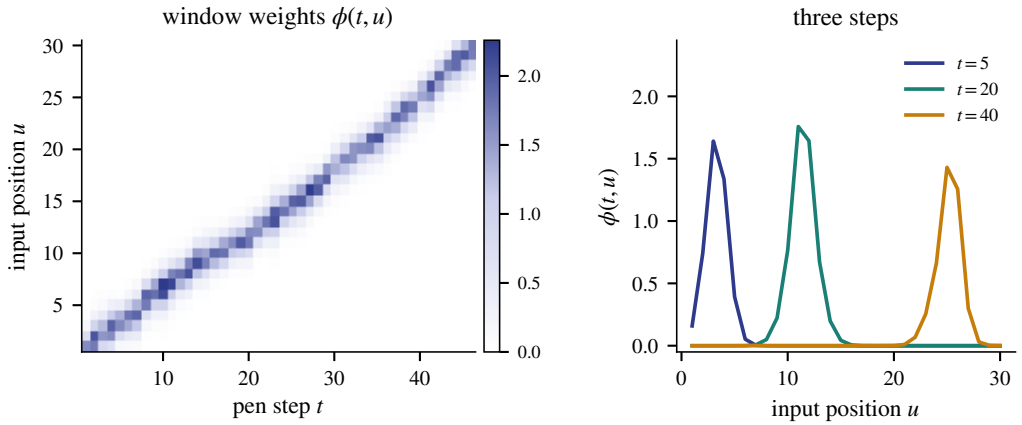


Figure 3. A Gaussian-mixture window (7) with $K = 2$ moving along $U = 30$ characters. Left: $\phi(t,u)$ for 46 steps with random positive offsets. Right: the window at three steps. The alignment is monotonic because the centres can only increase.

Algorithm 2 Gaussian-mixture window (one step)

Require: hidden state $\mathbf{h}_t$; previous centres $\boldsymbol{\kappa}_{t-1} \in \mathbb{R}^K$; character vectors $\boldsymbol{\chi}_1, \dots, \boldsymbol{\chi}_U$; output layer $W, \mathbf{b}$

function WindowStep($\mathbf{h}_t, \boldsymbol{\kappa}_{t-1}$)
  $(\hat{\boldsymbol{\alpha}}, \hat{\boldsymbol{\beta}}, \hat{\boldsymbol{\kappa}}) \leftarrow W\mathbf{h}_t + \mathbf{b}$ ▷ $3K$ numbers
  $\boldsymbol{\alpha} \leftarrow \exp(\hat{\boldsymbol{\alpha}})$; $\boldsymbol{\beta} \leftarrow \exp(\hat{\boldsymbol{\beta}})$; $\boldsymbol{\kappa}_t \leftarrow \boldsymbol{\kappa}_{t-1} + \exp(\hat{\boldsymbol{\kappa}})$ ▷ centres only move forward
  for $u = 1, \dots, U$ **do**
    $\phi_u \leftarrow \sum_{k=1}^{K} \alpha^k \exp\big(-\beta^k(\kappa_t^k - u)^2\big)$
  end for
  return $\mathbf{w}_t = \sum_u \phi_u \boldsymbol{\chi}_u$ and $\boldsymbol{\kappa}_t$
end function

Running time — Gaussian-mixture window

Time: $O(Kd_h)$ for the window parameters and $O(KU)$ for the weights per step; in practice only the few positions within a handful of widths of each centre matter, so the cost can be cut to $O(K)$ per step with truncation. Over T steps, $O(T(Kd_h + KU))$, far less than content-based attention when U is large. *Memory*: $O(U)$ for the weights per step, $O(TU)$ (or $O(TK)$ truncated) for training. The window adds only $3K(d_h + 1)$ parameters.

Variants. Content and location can be mixed. Chorowski et al. added to the content score a term computed by a convolution of the previous alignment $\boldsymbol{\alpha}_{i-1}$, so that the model knows where it looked last and prefers to move a little forward; this made attention-based speech recognition work on long utterances [9]. Other variants keep a running summary of past alignments in a small recurrent network, a generalisation of that convolution, or give every input position its own learned embedding so that keys can encode location, as memory networks do. A purely location-based window, on the other hand, cannot repair its mistakes: if a jump such as the one at $t = 4$ in Worked example 3 skips a character that still had to be written, it can never be undone, because the centres cannot move back.

3.2 Connectionist temporal classification

Connectionist temporal classification (CTC) [17] predates soft attention and is still widely used for speech and handwriting recognition. It solves the monotonic alignment problem exactly, by summing over all alignments.

Paths and the collapse map. An RNN (or any network) reads the T input frames and outputs at every frame a distribution y_k^t over the alphabet plus one extra symbol, the *blank* "–". A sequence $\pi = (\pi_1, \dots, \pi_T)$ of such symbols is a *path*. The collapse map $\mathcal{B}$ first merges repeated symbols and then deletes blanks: $\mathcal{B}(\texttt{aa-b}) = \mathcal{B}(\texttt{a-bb}) = \texttt{ab}$, while $\mathcal{B}(\texttt{a-a}) = \texttt{aa}$ — a blank is needed between two equal letters. The labelling $\mathbf{l}$ (which needs $T \geq |\mathbf{l}| + r$, where r is the number of adjacent equal labels in $\mathbf{l}$) gets the probability

$$p(\mathbf{l} \mid \mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} p(\pi \mid \mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} \prod_{t=1}^{T} y_{\pi_t}^t. \tag{9}$$

Each path is a monotonic *hard* alignment of the label to the frames, and CTC marginalises over all of them. The product form assumes that the symbols at different frames are conditionally independent given $\mathbf{x}$; the dependence between output symbols comes only through the shared input. The RNN transducer of Graves removes this assumption by feeding the previously emitted labels to a prediction network while keeping the monotonic sum over alignments [18]. Training maximises $\log p(\mathbf{l} \mid \mathbf{x})$ (maximum likelihood, Note 2, Section 2.1).

The forward recursion. The number of paths grows exponentially with T, but monotonicity allows dynamic programming. Insert blanks around and between the labels, $\mathbf{l}' = (-, l_1, -, l_2, \dots, l_{|\mathbf{l}|}, -)$, of length $S = 2|\mathbf{l}| + 1$, and let $\gamma_t(s)$ be the total probability of all path prefixes of length t that end at position s of $\mathbf{l}'$ and are consistent with it. A path may stay on the same symbol, advance by one, or skip a blank between two *different* labels:

$$\gamma_t(s) = y_{l'_s}^t \Big(\gamma_{t-1}(s) + \gamma_{t-1}(s-1) + \mathbf{1}\{l'_s \neq - \text{ and } l'_s \neq l'_{s-2}\}\, \gamma_{t-1}(s-2) \Big), \tag{10}$$

with $\gamma_1(1) = y_{-}^1$, $\gamma_1(2) = y_{l_1}^1$, and $\gamma_1(s) = 0$ otherwise. A complete path ends on the last label or on the final blank: $p(\mathbf{l} \mid \mathbf{x}) = \gamma_T(S) + \gamma_T(S-1)$. A backward recursion of the same form gives the gradient, as in the forward–backward algorithm of hidden Markov models.

Worked example — 4. CTC on three frames

Alphabet $\{\texttt{a}, \texttt{b}\}$ plus blank, $T = 3$ frames, label $\mathbf{l} = \texttt{ab}$. The network outputs

frame	$y_{\texttt{a}}$	$y_{\texttt{b}}$	y_{-}
1	0.6	0.1	0.3
2	0.2	0.3	0.5
3	0.1	0.6	0.3

By enumeration. Five of the 27 paths collapse to ab: aab ($0.6 \cdot 0.2 \cdot 0.6 = 0.072$), abb (0.108), ab– (0.054), a–b (0.180) and –ab (0.036); the total is $p(\texttt{ab} \mid \mathbf{x}) = 0.450$. *By the recursion* on $\mathbf{l}' = (-, \texttt{a}, -, \texttt{b}, -)$:

	–	a	–	b	–
γ_1	0.300	0.600	0	0	0
γ_2	0.150	0.180	0.300	0.180	0
γ_3	0.045	0.033	0.144	0.396	0.054

For instance $\gamma_2(\texttt{b}) = 0.3\,(0 + 0 + 0.6) = 0.18$ uses the skip from a over the blank, and $\gamma_3(\texttt{b}) = 0.6\,(0.18 + 0.30 + 0.18) = 0.396$. The answer $\gamma_3(4) + \gamma_3(5) = 0.396 + 0.054 = 0.450$ agrees. The label aa has only one path of length three, a–a, with probability $0.6 \cdot 0.5 \cdot 0.1 = 0.03$.

Algorithm 3 CTC forward pass

Require: per-frame distributions y^t over the alphabet and blank, $t = 1, \dots, T$; label $\mathbf{l}$

function CTCForward($y, \mathbf{l}$)
  $\mathbf{l}' \leftarrow (-, l_1, -, \dots, l_{|\mathbf{l}|}, -)$; $S \leftarrow 2|\mathbf{l}| + 1$
  $\gamma_1(1) \leftarrow y^1_-$; $\gamma_1(2) \leftarrow y^1_{l_1}$; $\gamma_1(s) \leftarrow 0$ for $s > 2$
  for $t = 2, \dots, T$ **do**
    for $s = 1, \dots, S$ **do**
      $b \leftarrow \gamma_{t-1}(s) + \gamma_{t-1}(s-1)$ ▷ stay or advance; $\gamma(0) = 0$
      if $l'_s \neq -$ and $s > 2$ and $l'_s \neq l'_{s-2}$ **then** $b \leftarrow b + \gamma_{t-1}(s-2)$ ▷ skip a blank
      end if
      $\gamma_t(s) \leftarrow y^t_{l'_s}\, b$
    end for
  end for
  return $\log\big(\gamma_T(S) + \gamma_T(S-1)\big)$ ▷ in practice: work with $\log\gamma$ and log-sum-exp
end function

Running time — CTC loss

Time: $O(TS) = O(T|\mathbf{l}|)$ for the forward recursion and the same for the backward one, negligible next to the network that produces the y^t. *Memory*: $O(TS)$ for the table, needed by the backward pass. Products of many probabilities underflow, so implementations work with logarithms (the log-sum-exp trick of Note 5, Section 3.3) or rescale each row. *Prediction*: the most likely labelling has no closed form; taking the most likely symbol at every frame and collapsing (best-path decoding) costs $O(TV)$, and a prefix beam search with width B is more accurate.

Comparison. The three mechanisms differ in what they assume. Content-based weights assume nothing about the order of the alignment; a location-based window assumes that it is monotonic; CTC assumes in addition that the outputs are conditionally independent, and in exchange computes the likelihood exactly. Each path of CTC is a hard alignment, one label position per frame, yet the gradient is exact because the paths are summed rather than sampled: Worked example 4 computed exactly the sum that the sampling estimator of Section 4.2 would have to approximate.

Modern connection — speech recognition today

Both lines are alive. Self-supervised speech encoders such as wav2vec 2.0 are fine-tuned for recognition with a CTC output layer [3], which keeps decoding simple and strictly monotonic. Whisper [37] instead uses a transformer encoder–decoder of the kind assembled in Section 5: the encoder reads a log-Mel spectrogram, and the decoder attends to it with the cross-attention of Section 5.4 while generating text; trained on a very large collection of weakly labelled audio, it transcribes and translates many languages.

4 Hard attention and glimpses

4.1 A recurrent model of visual attention

A convolutional network (Note 13) processes the whole image at full resolution, and its cost grows with the number of pixels. The eye works differently: only the fovea, about two degrees of the visual field, is sharp, so we scan a scene with a few saccades per second and combine what each fixation reveals. The *recurrent attention model* (RAM) of Mnih, Heess, Graves and Kavukcuoglu [31]

imitates this. A *glimpse sensor* extracts, around a location $\mathbf{l}_{t-1}$, a few patches of increasing size, all reduced to the same small number of pixels, so that the centre is seen sharply and the surround coarsely. A *glimpse network* maps them and the location to features $\mathbf{g}_t$, and a *core* RNN updates its state $\mathbf{h}_t = f_h(\mathbf{h}_{t-1}, \mathbf{g}_t)$. A *location network* outputs the mean $\boldsymbol{\mu}_t = f_l(\mathbf{h}_t)$ of a Gaussian from which the next location is *sampled*, $\mathbf{l}_t \sim \mathcal{N}(\boldsymbol{\mu}_t, \sigma^2 I)$, and an *action network* $f_a(\mathbf{h}_t)$ produces the output, for example a class after the last glimpse. The cost per glimpse is independent of the image size. This is *hard* attention: at each step exactly one location is chosen.

The objective. Write $\mathbf{z}_{1:T}$ for the interaction sequence (glimpses, locations and actions) and r_t for the reward received at step t, for instance $r_T = 1$ if the final classification is correct and 0 otherwise. The policy $\pi(\cdot \mid \mathbf{z}_{1:t}; \theta)$ is the distribution of the next location (and action) given the past. The model maximises the expected total reward

$$J(\theta) = \mathbb{E}_{p(\mathbf{z}_{1:T};\theta)}\Big[\sum_{t=1}^{T} r_t\Big] = \mathbb{E}_{p(\mathbf{z}_{1:T};\theta)}[R]. \tag{11}$$

The sampling of $\mathbf{l}_t$ is not differentiable — a small change of θ changes the probability of a location, not the location itself — so backpropagation alone cannot compute ∇J.

A likelihood-based variant. For classification the correct class y^* is known, and the reward can be replaced by the log-probability the action network assigns to it after the glimpses. Then

$$\mathbb{E}_{\mathbf{l}}\big[\log p(y^* \mid \mathbf{l}, \mathbf{x})\big] \le \log \mathbb{E}_{\mathbf{l}}\big[p(y^* \mid \mathbf{l}, \mathbf{x})\big] = \log p(y^* \mid \mathbf{x})$$

by Jensen's inequality: this maximises a lower bound on the log-likelihood of the correct class, with the glimpse locations as latent variables [2].

4.2 The REINFORCE estimator

Let $p_\theta(z)$ be any distribution over discrete or continuous outcomes z (a sequence of locations) and $R(z)$ a reward that does not depend on θ. Then

$$\nabla_\theta \mathbb{E}_{p_\theta}[R] = \nabla_\theta \sum_z p_\theta(z) R(z) = \sum_z p_\theta(z)\, \frac{\nabla_\theta p_\theta(z)}{p_\theta(z)}\, R(z) = \mathbb{E}_{p_\theta}\big[R(z)\, \nabla_\theta \log p_\theta(z)\big], \tag{12}$$

the *log-derivative trick* (sums become integrals for continuous z). The right-hand side is an expectation, so it can be estimated by sampling: draw M episodes $z^{(m)}$ from the current policy and average $R(z^{(m)}) \nabla_\theta \log p_\theta(z^{(m)})$. This is Williams' REINFORCE estimator [46]. For a sequence of choices, $\log p_\theta(z) = \sum_t \log \pi(\mathbf{l}_t \mid \mathbf{z}_{1:t}; \theta)$ plus terms that do not depend on θ, and each term is differentiated by backpropagation through the RNN. For the Gaussian location policy, $\nabla_{\boldsymbol{\mu}} \log \mathcal{N}(\mathbf{l}; \boldsymbol{\mu}, \sigma^2 I) = (\mathbf{l} - \boldsymbol{\mu})/\sigma^2$: the estimator moves the mean towards locations that earned a high reward.

Baselines. The estimator is unbiased but very noisy. Because $\sum_z p_\theta(z) = 1$ for every θ,

$$\mathbb{E}_{p_\theta}[\nabla_\theta \log p_\theta(z)] = \sum_z \nabla_\theta p_\theta(z) = \nabla_\theta 1 = 0,$$

so any constant b, the *baseline*, can be subtracted from the reward without changing the expectation: $\nabla J = \mathbb{E}[(R - b) \nabla \log p_\theta]$. A good baseline, close to the expected reward, makes $R - b$ small and of either sign and can reduce the variance enormously. Mnih et al. learn a baseline b_t that depends on the state, by regressing it onto the observed rewards.

Worked example — 5. Two possible glimpses

A policy looks left with probability $\pi = \sigma(\theta)$ and right otherwise; the object is on the left. With rewards $R = 1$ (left) and 0 (right), $J = \pi$ and $\partial J/\partial\theta = \pi(1-\pi) = 0.25$ at $\theta = 0$. The score function is $\partial_\theta \log \pi = 1 - \pi = 0.5$ for left and $\partial_\theta \log(1-\pi) = -\pi = -0.5$ for right. A single-sample estimate is therefore $1 \cdot 0.5 = 0.5$ or $0 \cdot (-0.5) = 0$, each with probability $\frac{1}{2}$: mean 0.25, variance 0.0625.

Now shift the rewards to $R = 11$ and 10, which leaves the problem and the true gradient unchanged. The estimates become $11 \cdot 0.5 = 5.5$ and $10 \cdot (-0.5) = -5$: still mean 0.25, but variance 27.6, more than four hundred times larger. With the baseline $b = \mathbb{E}[R] = 10.5$ they become $(0.5)(0.5) = 0.25$ and $(-0.5)(-0.5) = 0.25$: zero variance in this example.

Figure 4 repeats the experiment with five possible locations, rewards 10 or 11 and mini-batches of eight episodes. Without a baseline the estimates scatter with a standard deviation ten times larger than the gradient itself; with the baseline they cluster around it.

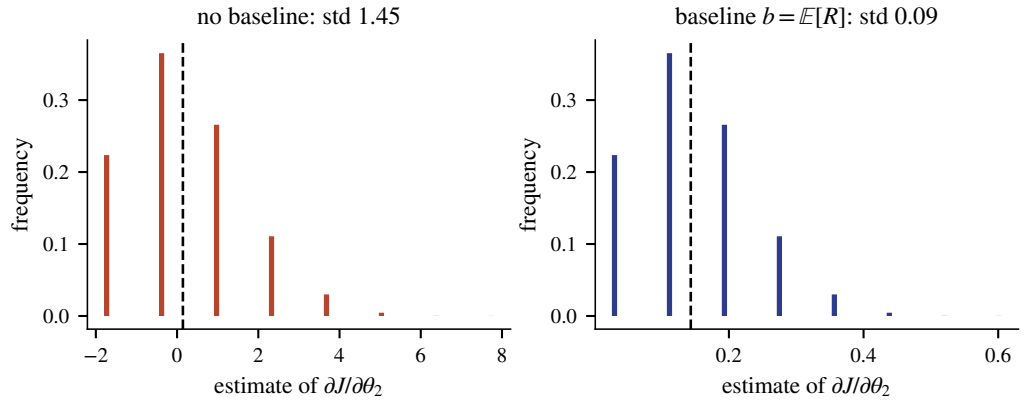


Figure 4. Distribution of 20 000 REINFORCE estimates of $\partial J/\partial\theta_2$ for a softmax policy over five locations ($\boldsymbol{\theta} = (0.4, 0, -0.2, 0.1, 0.3)$, the object at location 2, reward $10 + \mathbf{1}\{\text{location } 2\}$), each averaged over $M = 8$ episodes. Dashed line: the exact gradient, 0.143. The standard deviation drops from 1.45 to 0.09 with the baseline $b = \mathbb{E}[R]$.

Algorithm 4 REINFORCE with a baseline for a recurrent attention model

Require: image $\mathbf{x}$ and label y^*; policy and classifier with parameters θ; baseline $b_\psi(\mathbf{h})$; M episodes of T glimpses; learning rates

function RAMUPDATE($\mathbf{x}, y^*$)
  for $m = 1, \dots, M$ **do**
    $\mathbf{h}_0 \leftarrow \mathbf{0}$; $\mathbf{l}_0 \leftarrow$ initial location
    for $t = 1, \dots, T$ **do**
      $\mathbf{g}_t \leftarrow$ glimpse network on the patches around $\mathbf{l}_{t-1}$; $\mathbf{h}_t \leftarrow f_h(\mathbf{h}_{t-1}, \mathbf{g}_t)$
      $\mathbf{l}_t \sim \mathcal{N}(f_l(\mathbf{h}_t), \sigma^2 I)$ ▷ sampled: no gradient through this step
    end for
    $R^{(m)} \leftarrow \mathbf{1}\{\arg\max f_a(\mathbf{h}_T) = y^*\}$ ▷ reward of the episode
  end for
  $\mathbf{g}_\theta \leftarrow \frac{1}{M} \sum_m \sum_t \left(R^{(m)} - b_\psi(\mathbf{h}_t^{(m)})\right) \nabla_\theta \log \mathcal{N}\left(\mathbf{l}_t^{(m)}; f_l(\mathbf{h}_t^{(m)}), \sigma^2 I\right)$
  $\mathbf{g}_\theta \leftarrow \mathbf{g}_\theta + \frac{1}{M} \sum_m \nabla_\theta \log p_\theta\left(y^* \mid \mathbf{h}_T^{(m)}\right)$ ▷ backprop for the classifier
  $\mathbf{g}_\psi \leftarrow \frac{1}{M} \sum_{m,t} \nabla_\psi \left(R^{(m)} - b_\psi(\mathbf{h}_t^{(m)})\right)^2$ ▷ fit the baseline
  $\theta \leftarrow \theta + \eta\, \mathbf{g}_\theta$; $\psi \leftarrow \psi - \eta'\, \mathbf{g}_\psi$ ▷ ascent on J, descent on the baseline loss
end function

Table 1. Soft and hard attention.

	soft (content-based, Section 2)	hard (RAM, this section)
what is learned	a scoring function	a sampling policy
output of the mechanism	weights on all positions	one sampled position
gradient	exact, by backpropagation	estimated by REINFORCE (Algorithm 4)
gradient variance	none	high, reduced by a baseline (Figure 4)
cost per step	O(input size)	$O(1)$ in the input size
memory for backprop	all weights	only the sampled path
examples in this note	Algorithm 1, Show, attend and tell, DRAW	RAM (Algorithm 4)

Running time — recurrent attention model with REINFORCE

Time per update: MT glimpses, each costing the glimpse network on a few small patches of P pixels, $O(Pd_g)$, and an RNN step, $O(d_h^2 + d_h d_g)$, plus the same again for the backward pass. None of this depends on the image size. *Memory*: $O(MT(d_h + d_g))$ for the episode states. *Sample efficiency* is the hidden cost: the variance of the estimator means many more updates than for a model trained by backpropagation alone. *Prediction*: one episode of T glimpses.

4.3 Soft versus hard, and soft attention for images

Table 1 compares the two kinds of attention.

The two kinds are not tied to their first applications. Soft content-based attention works on images: in *Show, attend and tell* [47] a convnet turns an image into a 14×14 grid of feature vectors, and an LSTM that writes a caption attends over the 196 positions at each word, exactly as in (3)–(4); the same paper also trained a stochastic hard version. Conversely, hard attention has been used for sequence-to-sequence tasks: Luo et al. trained a model that decides online, by sampling, when to emit an output token, with the REINFORCE estimator of Section 4.2; it works, but needs careful

variance reduction to train stably, as the analysis of that section leads one to expect [29].

DRAW: soft location-based attention for images. The DRAW network of Gregor et al. [20] generates images by repeatedly reading from and writing to a canvas through a differentiable window: an $N \times N$ grid of Gaussian filters whose centre (g_X, g_Y), stride δ, width σ and intensity γ the network emits at each step. Filter i along x is centred at $\mu_X^i = g_X + (i - N/2 - 1/2)\,\delta$; the matrix $F_X \in \mathbb{R}^{N\times A}$ has row-normalised entries proportional to $\exp\big(-(a-\mu_X^i)^2/(2\sigma^2)\big)$, and similarly F_Y. The glimpse of an image $\mathbf{X} \in \mathbb{R}^{B\times A}$ (B rows, A columns; $F_Y \in \mathbb{R}^{N\times B}$) is the patch $\gamma F_Y \mathbf{X} F_X^{\mathsf{T}}$: a small stride zooms in, a large one sees the whole image coarsely. Like Graves' window it is location-based and soft, so plain backpropagation trains it.

5 Self-attention and the transformer

The attention of Section 2 connects a decoder to an encoder that is still recurrent. Vaswani et al. [43] showed that attention can also be used *inside* the encoder and the decoder, replacing recurrence and convolution entirely. The resulting *transformer* became the standard architecture for language, and increasingly for other data.

5.1 Generalised dot-product attention

Abstract from the decoder and the encoder. A *query* $\mathbf{q} \in \mathbb{R}^{d_k}$ is compared with T_k *keys* $\mathbf{k}_1, \dots, \mathbf{k}_{T_k} \in \mathbb{R}^{d_k}$, and the result is a weighted average of T_k *values* $\mathbf{v}_1, \dots, \mathbf{v}_{T_k} \in \mathbb{R}^{d_v}$:

$$A(\mathbf{q}; K, V) = \sum_{j=1}^{T_k} \frac{\exp(\mathbf{q}^{\mathsf{T}}\mathbf{k}_j/\sqrt{d_k})}{\sum_{m=1}^{T_k}\exp(\mathbf{q}^{\mathsf{T}}\mathbf{k}_m/\sqrt{d_k})}\,\mathbf{v}_j. \tag{13}$$

In Section 2 the query was the decoder state and keys and values were both the encoder states; separating keys from values lets a position be found by one description and deliver another. Note 5, Section 3.3 (modern box), read (13) as a Nadaraya–Watson smoother with an exponential kernel whose queries, keys and values are learned (Section 5.2). Stacking T_q queries as the rows of $Q \in \mathbb{R}^{T_q\times d_k}$, and keys and values as the rows of $K \in \mathbb{R}^{T_k\times d_k}$ and $V \in \mathbb{R}^{T_k\times d_v}$, all queries are answered at once:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\Big(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\Big)V \in \mathbb{R}^{T_q\times d_v}, \tag{14}$$

with the softmax taken along each row (V is not the vocabulary here). Two matrix products and a row-wise softmax: exactly the operations that GPUs do best.

Derivation: why divide by $\sqrt{d_k}$. Suppose that at initialisation the components of $\mathbf{q}$ and $\mathbf{k}$ are independent with mean 0 and variance 1, as they roughly are when the inputs are layer-normalised and the projections use a variance-preserving initialisation. Then $\mathbf{q}^{\mathsf{T}}\mathbf{k} = \sum_{a=1}^{d_k} q_a k_a$ is a sum of d_k independent terms, each with mean $\mathbb{E}[q_a]\mathbb{E}[k_a] = 0$ and variance

$$\mathrm{Var}(q_a k_a) = \mathbb{E}[q_a^2 k_a^2] - (\mathbb{E}[q_a k_a])^2 = \mathbb{E}[q_a^2]\,\mathbb{E}[k_a^2] - 0 = 1,$$

so $\mathrm{Var}(\mathbf{q}^{\mathsf{T}}\mathbf{k}) = d_k$ and the scores have standard deviation $\sqrt{d_k}$ (the computation behind Glorot and He initialisation, Note 12, Section 7). For $d_k = 64$ the scores have standard deviation 8 and two scores typically differ by about 11; the softmax of such scores is nearly one-hot, and by (6) its gradient

nearly vanishes. Dividing by $\sqrt{d_k}$ restores unit variance whatever the width; in the language of the physics box of Section 2.5, it sets the temperature to $\sqrt{d_k}$. Figure 5 measures the effect for 32 random keys: without scaling the entropy of the weights collapses as d_k grows and the median Jacobian of the softmax falls by two orders of magnitude at $d_k = 512$; with scaling both stay constant.

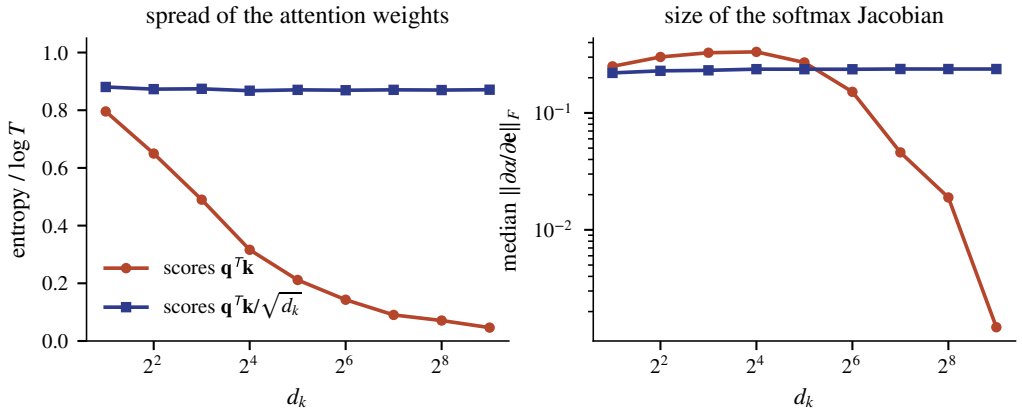


Figure 5. Softmax over $T = 32$ keys with standard normal queries and keys, over 2000 draws. Left: mean entropy of the weights divided by its maximum $\log T$. Right: median Frobenius norm of the Jacobian $\mathrm{diag}(\boldsymbol{\alpha}) - \boldsymbol{\alpha}\boldsymbol{\alpha}^{\mathsf{T}}$. With unscaled scores the entropy ratio falls to 0.14 at $d_k = 64$ and to 0.05 at $d_k = 512$, and the median Jacobian to 0.0015 at $d_k = 512$; with scaled scores both stay near 0.87 and 0.24.

5.2 Self-attention and masks

In *self-attention* a sequence attends to itself. The rows of $X \in \mathbb{R}^{T\times d}$ are the representations of the T positions, and queries, keys and values are three learned linear maps of the same rows:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \qquad \text{output} = \mathrm{softmax}\Big(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\Big)V, \tag{15}$$

with $W_Q, W_K \in \mathbb{R}^{d\times d_k}$, $W_V \in \mathbb{R}^{d\times d_v}$, and a *mask* $M \in \{0, -\infty\}^{T\times T}$ added to the scores. An entry $-\infty$ gives the weight $e^{-\infty} = 0$, so it forbids position i to read position j. Two masks are common:

- the *causal* mask, $M_{ij} = -\infty$ for $j > i$, lets every position see only itself and the past. It is what makes a transformer a valid autoregressive model, $p(y_i \mid y_{<i})$, as in (1);
- the *padding* mask hides the filler positions added to make the sequences of a mini-batch equally long.

In a decoder the input sequence is the target shifted right by one, (<bos>, $y_1, \dots, y_{T'-1}$), so position i holds y_{i-1} and, with the causal mask, its output depends on $y_{<i}$ only; that output predicts y_i.

Worked example — 6. Self-attention on three tokens

Three tokens with $d = d_k = d_v = 2$ have the rows $\mathbf{x}_1 = (1, 0)$, $\mathbf{x}_2 = (0, 1)$, $\mathbf{x}_3 = (1, 1)$ and

$$W_Q = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad W_K = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad W_V = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \quad \text{so that}$$

$$Q = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 2 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 2 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 2 \end{pmatrix}.$$

Scores. QK^{T} has the rows $(1, 2, 3)$, $(0, 1, 1)$, $(1, 3, 4)$; divided by $\sqrt{2}$ they become

$(0.707, 1.414, 2.121)$, $(0, 0.707, 0.707)$, $(0.707, 2.121, 2.828)$. *Weights.* Row 1 has the scores of Worked example 1, so its weights are the scaled-dot weights found there; rows 2 and 3 follow in the same way:

$$A = \begin{pmatrix} 0.140 & 0.284 & 0.576 \\ 0.198 & 0.401 & 0.401 \\ 0.074 & 0.306 & 0.620 \end{pmatrix}, \qquad AV = \begin{pmatrix} 0.716 & 1.720 \\ 0.599 & 1.604 \\ 0.694 & 1.851 \end{pmatrix}.$$

Token 2 attends equally to tokens 2 and 3 because their keys give it the same score. *Causal mask.* Setting the scores above the diagonal to $-\infty$ leaves row 1 with the single weight 1 (output $\mathbf{v}_1 = (1, 0)$), row 2 with $\mathrm{softmax}(0, 0.707) = (0.330, 0.670)$ (output $(0.330, 1.340)$), and row 3 unchanged. The first token now knows nothing about its successors, which is what a model that predicts the next token requires.

In physics — permutation symmetry and the need for positions

Let P be a $T \times T$ permutation matrix, so that PX lists the same tokens in another order. Then $(PX)W_Q = PQ$, similarly for K and V, and

$$\begin{aligned} \mathrm{softmax}\left(PQ(PK)^{\mathsf{T}}/\sqrt{d_k}\right)PV &= \mathrm{softmax}\left(P\,QK^{\mathsf{T}}P^{\mathsf{T}}/\sqrt{d_k}\right)PV \\ &= P\ \mathrm{softmax}\left(QK^{\mathsf{T}}/\sqrt{d_k}\right)P^{\mathsf{T}}PV = P\left(\mathrm{output}\right), \end{aligned}$$

because a row-wise softmax commutes with permuting rows and columns, and $P^{\mathsf{T}}P = I$. Self-attention without a mask is *permutation equivariant*: it treats its input as a set of interacting particles, much like a system of identical particles with pairwise (though not necessarily reciprocal) interactions, whose dynamics does not care how the particles are labelled. For particles in a jet, atoms in a molecule or galaxies in a survey this is the right symmetry, and transformers without positional information are natural models for them. For language (as for Note 1's bag of words) it is wrong: "dog bites man" and "man bites dog" would get the same representations, permuted. The order must be supplied explicitly (Section 5.5). A causal mask also breaks the symmetry, because it singles out an order.

5.3 Multi-head attention

One attention operation produces, at each position, one weighted average: one way of choosing what to read. Multi-head attention runs h attention operations in parallel, each in its own lower-dimensional subspace, and concatenates the results:

$$\mathrm{head}_r = \mathrm{Attention}\big(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}\big), \qquad \mathrm{MHA}(X) = [\mathrm{head}_1, \dots, \mathrm{head}_h]\, W_O, \tag{16}$$

with $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{d \times d/h}$ and $W_O \in \mathbb{R}^{d \times d}$. One head may track the previous token, another the subject of the verb, a third the matching bracket. With $d_k = d_v = d/h$ the number of parameters, $4d^2$ plus $4d$ biases, and the number of multiply–adds do not depend on h (only the hT^2 softmax evaluations and stored weights grow with h): the heads split the width, they do not multiply it.

Algorithm 5 Multi-head (masked) self-attention, forward pass

Require: $X \in \mathbb{R}^{T\times d}$; h heads with $d_k = d/h$; $W_Q, W_K, W_V, W_O \in \mathbb{R}^{d\times d}$ (the head blocks side by side) and biases; mask M (causal, padding, or none)

function MultiHeadAttention(X; M)
  $Q \leftarrow XW_Q,\ K \leftarrow XW_K,\ V \leftarrow XW_V$ ▷ three $T \times d$ products, $O(Td^2)$ each
  for $r = 1, \dots, h$ **do** ▷ all heads in parallel in practice
    $Q_r, K_r, V_r \leftarrow$ columns $(r-1)d_k + 1, \dots, rd_k$ of Q, K, V
    $S_r \leftarrow Q_r K_r^{\mathsf{T}}/\sqrt{d_k} + M$ ▷ $T \times T$ scores, $O(T^2 d_k)$
    $A_r \leftarrow$ row-wise softmax of S_r (subtract each row's maximum first)
    $H_r \leftarrow A_r V_r$ ▷ $O(T^2 d_k)$
  end for
  return $[H_1, \dots, H_h]\, W_O$ ▷ $O(Td^2)$
end function

Running time — multi-head self-attention

Time: $4Td^2$ multiply–adds for the four projections and $2T^2d$ for scores and weighted sums (summed over the h heads, $h \cdot 2T^2 d_k = 2T^2 d$), so $O(Td^2 + T^2 d)$ per layer and sequence; the second term dominates once $T > 2d$ when only attention is counted. *Memory*: $O(Td)$ for the activations and $O(hT^2)$ for the attention weights, which a standard implementation stores for the backward pass. *Parallelism*: unlike an RNN, all T positions are processed at once; the sequential depth of a layer is $O(1)$, and the path between any two positions has length one. For comparison, a recurrent layer costs $O(Td^2)$ but needs T sequential steps, and the path between positions i and j has length $|i - j|$ (Note 14, Section 2). *Backward*: about twice the forward.

5.4 Three kinds of attention in the transformer

The encoder–decoder transformer of Vaswani et al. uses attention in three places.

1. *Encoder self-attention*: Q, K and V all come from the encoder's representation of the source sentence, with only a padding mask. Every source position can read every other one; this replaces the bidirectional RNN.
2. *Masked decoder self-attention*: Q, K and V come from the decoder's representation of the target prefix, with a causal mask. This replaces the decoder RNN. During training, with teacher forcing (Note 14, Section 7), all target positions are computed in parallel in one pass, the shifted input and the mask guaranteeing that the output at position i, which predicts y_i, never depends on $y_i, y_{i+1}, \dots$
3. *Encoder–decoder (cross-) attention*: queries from the decoder, keys and values from the final encoder output. This is the attention of Section 2, with a scaled dot-product score instead of the additive one.

5.5 Positional information

Because self-attention is permutation equivariant, the transformer adds a vector $\mathbf{p}_t \in \mathbb{R}^d$ that depends only on the position t to the embedding of the token at position t. The original choice is a set of sinusoids of geometrically spaced frequencies,

$$p_{t,2i} = \sin(\omega_i t), \qquad p_{t,2i+1} = \cos(\omega_i t), \qquad \omega_i = 10\,000^{-2i/d}, \quad i = 0, \dots, d/2 - 1, \tag{17}$$

with wavelengths from 2π up to about $2\pi \cdot 10^4$ positions (Figure 6). *Learned* position embeddings, one trainable vector per position, work about as well in translation [43] but cannot be used beyond the longest training position.

Derivation: relative positions are linear. For one frequency ω, the addition theorems give

$$\begin{pmatrix} \sin \omega(t+k) \\ \cos \omega(t+k) \end{pmatrix} = \begin{pmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{pmatrix} \begin{pmatrix} \sin \omega t \\ \cos \omega t \end{pmatrix},$$

so $\mathbf{p}_{t+k} = R_k \mathbf{p}_t$ with a block-diagonal rotation R_k that depends on the offset k but not on t. A linear map can therefore express "the position k steps back", and the dot product

$$\mathbf{p}_t^{\mathsf{T}} \mathbf{p}_{t+k} = \sum_i \big[\sin \omega_i t \, \sin \omega_i (t+k) + \cos \omega_i t \, \cos \omega_i (t+k) \big] = \sum_i \cos \omega_i k$$

depends on the offset only, as the right panel of Figure 6 confirms (the three curves agree to 10^{-14}).

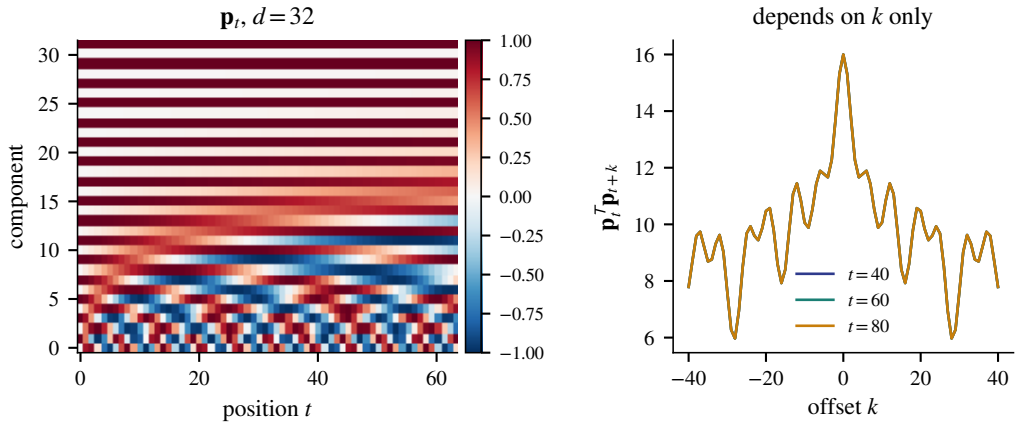


Figure 6. Left: the sinusoidal encodings (17) for $d = 32$ and the first 64 positions; low components oscillate fast, high ones slowly, like the digits of a continuous binary counter. Right: $\mathbf{p}_t^{\mathsf{T}} \mathbf{p}_{t+k}$ as a function of the offset k for three starting positions t; the curves coincide, with maximum $d/2 = 16$ at $k = 0$.

Modern connection — rotary position embeddings

Instead of adding positions to the input, *rotary position embeddings* (RoPE) [41] rotate each pair of components of the query at position m by the angle $m\omega_i$ and the key at position n by $n\omega_i$, with the frequencies of (17). Rotations are orthogonal, so $(R_m \mathbf{q})^{\mathsf{T}} (R_n \mathbf{k}) = \mathbf{q}^{\mathsf{T}} R_m^{\mathsf{T}} R_n \mathbf{k} = \mathbf{q}^{\mathsf{T}} R_{n-m} \mathbf{k}$: the score depends on the content and on the *relative* position $n - m$ only, in every layer. Many current open language models use RoPE, and several methods that extend their context length work by rescaling its frequencies.

5.6 The block and the full model

A transformer layer combines self-attention, which mixes information *between* positions, with a position-wise MLP, which processes each position separately with the same weights (Note 11, Section 4: one hidden layer of width d_{ff}, usually $4d$, with ReLU or GELU). Each of the two sublayers is wrapped in a residual connection and a layer normalisation, whose purpose and derivation are those of Note 12, Sections 10.2 and 10.3. In the *pre-normalisation* form, whose stability advantage over the original post-normalisation is discussed in Note 12, Section 10.3 (modern box),

$$X \leftarrow X + \mathrm{MHA}\big(\mathrm{LN}(X)\big), \qquad X \leftarrow X + \mathrm{MLP}\big(\mathrm{LN}(X)\big). \tag{18}$$

The full model of Vaswani et al. adds positional encodings to token embeddings (scaled by $\sqrt{d}$), stacks $N = 6$ encoder layers and $N = 6$ decoder layers (each decoder layer with masked self-attention, cross-attention and an MLP), and ends with a linear map to the vocabulary and a softmax that shares its weight matrix with the embeddings. The base model ($d = 512$, $h = 8$, $d_{\text{ff}} = 2048$) trained for about twelve hours on eight GPUs and already outperformed all previously published models on English–German translation, and a larger version set a new single-model record on English–French [43]; training was much faster than for recurrent models because no step waits for the previous one.

Algorithm 6 Transformer layer (pre-normalisation), encoder or decoder

Require: $X \in \mathbb{R}^{T \times d}$; for a decoder layer also the encoder output $Z \in \mathbb{R}^{T_{\text{src}} \times d}$; masks

function TransformerLayer(X; Z)
  $X \leftarrow X + \text{MultiHeadAttention}(\text{LN}_1(X); M)$ ▷ Algorithm 5; M causal in a decoder
  if decoder of an encoder–decoder model **then**
    $X \leftarrow X + \text{CrossAttention}\big(\text{LN}_2(X), Z\big)$ ▷ queries from X, keys and values from Z
  end if
  $X \leftarrow X + \max(0, \text{LN}_3(X) W_1 + \mathbf{1}\mathbf{b}_1^{\top})\, W_2 + \mathbf{1}\mathbf{b}_2^{\top}$ ▷ position-wise MLP, $W_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$
  return X
end function

Running time — one transformer layer

Time: $4Td^2$ for the attention projections, $2T^2d$ for scores and mixing, and $2Td d_{\text{ff}} = 8Td^2$ for the MLP with $d_{\text{ff}} = 4d$: in total $12Td^2 + 2T^2d$ multiply–adds per layer. A decoder layer adds $2Td^2 + 2T_{\text{src}}d^2$ for the cross-attention projections (the latter once per source sentence) and $2TT_{\text{src}}d$ for its scores and mixing. *Memory*: $12d^2 + O(d)$ parameters per layer ($16d^2$ with cross-attention); activations $O(Td + Td_{\text{ff}})$ and attention weights $O(hT^2)$ per layer and sequence for the backward pass. *A model of L layers*: L times this; training costs about three forward passes per example.

Worked example — 7. Shapes and parameters of the base transformer

Take $d = 512$, $h = 8$ (so $d_k = 64$), $d_{\text{ff}} = 2048$ and a source sentence of $T = 20$ tokens. *Shapes in one encoder layer:* X is 20×512; Q, K, V are 20×512, split into eight 20×64 blocks; each head has a 20×20 weight matrix and a 20×64 output; the concatenation and W_O return 20×512; the MLP goes through 20×2048 and back to 20×512. Every layer maps $T \times d$ to $T \times d$, which is why layers can be stacked and the residual additions are possible. *Parameters:* attention $4(512^2 + 512) = 1\,050\,624$; MLP $512 \cdot 2048 + 2048 + 2048 \cdot 512 + 512 = 2\,099\,712$; two layer norms $2 \cdot 2 \cdot 512 = 2048$; so an encoder layer has $3\,152\,384$ parameters, two thirds of them in the MLP, as for GPT-2 in Note 11, Section 4.2; a decoder layer adds cross-attention and a third layer norm, $4\,204\,032$. Six of each give 44.1 million, and a shared embedding table for a vocabulary of about $37\,000$ tokens adds $37\,000 \times 512 = 18.9$ million, about 63 million in all; Vaswani et al. report 65 million for the base model. *Cost:* for $T = 20$ the attention scores are a negligible $2T^2d = 4.1 \times 10^5$ multiply–adds per layer against $12Td^2 = 6.3 \times 10^7$; for $T = 3072 = 6d$ the two are equal.

In physics — tokens as interacting particles

Read the rows $\mathbf{x}_1, \ldots, \mathbf{x}_T$ of X as the positions of T particles in $\mathbb{R}^d$ and the layer index as time. For a single head with $d_v = d$ and ignoring the layer norm, one self-attention sublayer with its

residual connection is

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \sum_j \frac{e^{\beta\, \mathbf{x}_i^{\mathsf{T}} B\, \mathbf{x}_j}}{\sum_m e^{\beta\, \mathbf{x}_i^{\mathsf{T}} B\, \mathbf{x}_m}} U \mathbf{x}_j, \qquad B = W_Q W_K^{\mathsf{T}}, \quad U = (W_V W_O)^{\mathsf{T}}, \quad \beta = 1/\sqrt{d_k}$$

(with h heads the update is a sum of h such terms). It is an explicit Euler step of a system in which each particle is displaced by a linear map U of a weighted average of all particles (itself included), with weights set by a pairwise interaction B; with layer normalisation the particles move on a sphere. When T is large the sum is an average over the empirical distribution of particles: a *mean-field* dynamics, of the same form as the Vlasov-type equations of plasma physics or models of flocking. Geshkovski et al. analyse transformers in exactly this way and find that, in simplified settings, the particles tend to form clusters as depth grows [15]. A related reading identifies the attention update with one step of a modern Hopfield network, which retrieves the stored pattern closest to a query [38].

6 Decoder-only models and efficient attention

6.1 Decoder-only language models

Drop the encoder and the cross-attention, keep the masked decoder: a stack of causal self-attention layers trained to predict the next token is a language model, $p(y_1, \dots, y_T) = \prod_i p(y_i \mid y_{<i})$, of the kind that Note 14, Sections 1 and 2, built from RNNs. The GPT models [35, 7] are of this type, and so are almost all current large language models; a translation, a question or a program is presented as a prefix (the *prompt*) and the answer generated as its continuation. Encoder-only models such as BERT [11] use unmasked self-attention and are trained to recover input tokens that were hidden by a special token (a different use of the word *mask*). Training computes the loss at all T positions in parallel (Section 5.4), whereas an RNN must compute its states one after the other, and it needs no labels, so very large text collections can be used. Text is generated one token at a time, by greedy choice, beam search or sampling with a temperature or a nucleus (Note 14, Section 8); the next subsection makes this generation efficient.

6.2 The key–value cache

At generation step t the new token must attend to all t previous positions in every layer. Their keys and values do not change when a token is appended — the causal mask guarantees that earlier positions never see later ones — so they are computed once and stored: the *key–value (KV) cache*. Each step then only computes the query, key and value of the *new* position. In Algorithm 7 the prefill stops one token short, so that the loop handles y_P and every position enters the cache exactly once.

Running time — generation with and without a KV cache

With the cache, step t costs $O\big(L(d^2 + td) + dV\big)$: the projections and MLP of one position and attention over t cached keys. Generating N tokens after a prompt of P costs $O\big(L(Pd^2 + P^2 d)\big)$ for the prefill plus $O\big(L(Nd^2 + N(P+N)d) + NdV\big)$. *Without the cache*, every step reruns the whole prefix, $O(L(td^2 + t^2 d))$, and N steps cost $O\big(L(N(P+N)d^2 + N(P+N)^2 d)\big)$. *Memory*: the cache holds $2LTd$ numbers for T cached tokens, per sequence; at long contexts it, not the weights, limits the batch size. Each step reads the whole cache and all weights while doing little arithmetic per number read: generation is limited by memory bandwidth.

Algorithm 7 Autoregressive generation with a key–value cache (decoder-only model)

Require: prompt $y_1, \dots, y_P$; L layers; output (unembedding) matrix $W_U \in \mathbb{R}^{V\times d}$; number of new tokens N; a decoding rule (greedy, beam, sampling)

function GENERATE($y_{1:P}$)
  run $y_1, \dots, y_{P-1}$ through all layers with the causal mask; keep $K^{(l)}, V^{(l)} \in \mathbb{R}^{(P-1)\times d}$ ▷ prefill
  for $t = P+1, \dots, P+N$ **do**
    $\mathbf{x} \leftarrow$ embedding of y_{t-1} plus position $t-1$ ▷ one row
    for $l = 1, \dots, L$ **do**
      $\mathbf{q}, \mathbf{k}, \mathbf{v} \leftarrow$ projections of $\mathrm{LN}(\mathbf{x})$; append $\mathbf{k}$ to $K^{(l)}$ and $\mathbf{v}$ to $V^{(l)}$ ▷ $O(d^2)$
      $\mathbf{x} \leftarrow \mathbf{x} + [\mathrm{softmax}(K_r^{(l)}\mathbf{q}_r/\sqrt{d_k})^{\mathsf{T}} V_r^{(l)}]_{r=1,\dots,h}\, W_O$ ▷ $O(td)$ over the cache
      $\mathbf{x} \leftarrow \mathbf{x} + \mathrm{MLP}(\mathrm{LN}(\mathbf{x}))$ ▷ $O(d^2)$
    end for
    $y_t \leftarrow$ decoding rule applied to $\mathrm{softmax}(W_U\,\mathrm{LN}(\mathbf{x}))$ ▷ $O(dV)$
  end for
  return $y_{P+1:P+N}$
end function

Worked example — 8. How large is the cache

A hypothetical model has $L = 32$ layers of width $d = 4096$ with $h = 32$ heads, stored in 16-bit numbers (two bytes). Per token the cache holds a key and a value of size d in every layer: $2\cdot 32\cdot 4096\cdot 2 = 524\,288$ bytes, half a mebibyte. A context of $T = 4096$ tokens needs $2^{19}\cdot 2^{12} = 2^{31}$ bytes = 2 GiB per sequence, and a batch of 16 such sequences 32 GiB. *Multi-query attention* [40] lets all 32 query heads share a single key and value head, which divides the cache by 32 (64 MiB); *grouped-query attention* [1] shares one key–value head among each group of query heads, for example 8 groups of 4, which divides it by 4 (0.5 GiB) with less loss of quality.

6.3 IO-aware exact attention

A standard implementation of (14) writes the $T \times T$ score matrix to the GPU's main memory, reads it back for the softmax, writes the weights, and reads them again for the product with V. For long sequences this traffic, not the arithmetic, dominates the time, and the $O(T^2)$ memory limits T. FlashAttention [10] computes exactly the same result without ever forming the full matrix: it loads blocks of Q, K and V into the small, fast on-chip memory and combines the partial results with an *online softmax*.

Derivation: merging two softmax blocks. Split the scores of one query into two blocks. For each block keep its maximum m, the sum $Z = \sum_j e^{s_j - m}$ (a partial partition function) and the unnormalised output $\mathbf{o} = \sum_j e^{s_j - m}\mathbf{v}_j$. With $m = \max(m_1, m_2)$, rescaling each block to the common maximum gives

$$Z = e^{m_1 - m}Z_1 + e^{m_2 - m}Z_2, \qquad \mathbf{o} = e^{m_1 - m}\mathbf{o}_1 + e^{m_2 - m}\mathbf{o}_2, \qquad \text{output} = \mathbf{o}/Z, \tag{19}$$

exactly the full softmax average, because $e^{m_b - m}e^{s_j - m_b} = e^{s_j - m}$. The blocks can therefore be processed one after the other, keeping only m, Z and $\mathbf{o}$ — the log-sum-exp trick of Note 5, Section 3.3, made incremental.

Worked example — 9. Online softmax in two blocks

One query has scores $\mathbf{s} = (1, 3, 2 \mid 5, 0)$ for values $(2, -1, 0 \mid 1, 4)$ (scalars for simplicity). *Block 1:* $m_1 = 3$, $Z_1 = e^{-2} + e^{0} + e^{-1} = 0.1353 + 1 + 0.3679 = 1.5032$, $o_1 = 2e^{-2} - 1 + 0 = -0.7293$. *Block 2 arrives:* $m = 5$, so the old quantities are multiplied by $e^{3-5} = 0.1353$: $Z = 0.2034 + e^{0} + e^{-5} = 1.2102$ and $o = -0.0987 + 1 + 4e^{-5} = 0.9282$. *Output:* $0.9282/1.2102 = 0.7670$, identical to the softmax average computed in one go.

Algorithm 8 Tiled attention with an online softmax (forward pass, one block of queries)

Require: a block Q_b of B_q queries; keys and values in blocks K_c, V_c of B_k rows, $c = 1, \dots, \lceil T/B_k \rceil$

function TiledAttention(Q_b)
  $\mathbf{m} \leftarrow -\infty$, $\mathbf{z} \leftarrow \mathbf{0}$ (one entry per query), $O \leftarrow 0 \in \mathbb{R}^{B_q \times d_v}$
  for $c = 1, \dots, \lceil T/B_k \rceil$ **do** ▷ with a causal mask, skip blocks entirely in the future
    load K_c, V_c into fast memory; $S \leftarrow Q_b K_c^{\mathsf{T}} / \sqrt{d_k}$ (plus mask) ▷ $B_q \times B_k$, never written out
    $\mathbf{m}' \leftarrow \max(\mathbf{m}, \text{row maxima of } S)$; $P \leftarrow \exp(S - \mathbf{m}')$ row-wise
    $\mathbf{z} \leftarrow e^{\mathbf{m}-\mathbf{m}'} \odot \mathbf{z} + \text{row sums of } P$; $O \leftarrow \operatorname{diag}(e^{\mathbf{m}-\mathbf{m}'})\, O + PV_c$; $\mathbf{m} \leftarrow \mathbf{m}'$
  end for
  return $\operatorname{diag}(\mathbf{z})^{-1} O$ and $\log \mathbf{z} + \mathbf{m}$ ▷ the latter lets the backward pass recompute P
end function

Running time — tiled (IO-aware) attention

Arithmetic: the same $O(T^2 d)$ multiply–adds as the standard algorithm, plus $O(T^2 d_v / B_k)$ operations to rescale the running outputs; the result is exact, not an approximation. *Extra memory*: $O(T)$ for $\mathbf{m}$ and $\mathbf{z}$ instead of $O(T^2)$ for the weights. *Memory traffic*: with fast memory of size S_{M} the number of reads and writes of main memory is $O(T^2 d^2 / S_{\mathrm{M}})$ against $O(Td + T^2)$ for the standard algorithm [10], a large saving for typical d and S_{M}. *Backward*: the weights are recomputed block by block from Q, K and the stored $\log \mathbf{z} + \mathbf{m}$ instead of being read from memory — more arithmetic, less traffic, and faster overall.

Modern connection — attention beyond text

The same block now processes most kinds of data once they are cut into tokens. The *vision transformer* cuts an image into 16×16 patches (a strided convolution, Note 13, Section 7) and runs a transformer encoder on the sequence of patches [12]. CLIP trains an image encoder and a text encoder so that matching image–caption pairs have large dot products, over hundreds of millions of pairs [36]. In *mixture-of-experts* layers the single MLP of each block is replaced by many expert MLPs, and a small router sends each token to the top one or two [39, 14]: the number of parameters grows while the cost per token stays nearly constant. And the quadratic cost of attention has revived recurrence: state-space models such as S4 and Mamba process long sequences in linear time with a learned linear recurrence [21, 22] (Note 14, Section 11).

In physics — transformers on physics data

In particle physics a jet is a variable-size *set* of particles, each with momentum, charge and identity. The Particle Transformer [34] treats the particles as tokens without positional encodings — so that it is permutation equivariant, as the physics demands — and adds to the attention scores a bias computed from physically motivated pairwise features such as the angular distance and invariant mass of each pair of particles; it improved jet tagging over earlier graph networks on a large simulated data set. In structural biology the attention-based network AlphaFold 2 predicts protein structures from sequences with high accuracy [26].

7 A first look inside a transformer

Mechanistic interpretability tries to find out *how* a trained network computes its predictions: which components carry which information, and which of them are actually used. This section introduces the vocabulary and the basic experimental tools, which the course project (weeks 7–14) applies to small models.

7.1 The residual stream

Write the pre-normalisation model (18) for one position i and number all sublayers — every attention layer and every MLP — as components $c = 1, \dots, 2L$. Each component reads the current vector $\mathbf{x}_i$, through a layer norm, and *adds* its output to it. Unrolling the additions,

$$\mathbf{x}_i^{\text{final}} = \mathbf{x}_i^{(0)} + \sum_{c=1}^{2L} \mathbf{o}_i^{(c)}, \qquad \text{logits}_i = W_U \, \text{LN}\left(\mathbf{x}_i^{\text{final}}\right), \tag{20}$$

where $\mathbf{x}_i^{(0)}$ is the token and position embedding and $\mathbf{o}_i^{(c)}$ the output of component c at position i. The vector $\mathbf{x}_i$ is called the *residual stream*: a shared d-dimensional channel from which every component reads and to which it writes, and from which the unembedding W_U reads the prediction at the end (Figure 7). An attention layer is the only component that moves information between positions; MLPs act on each position separately.

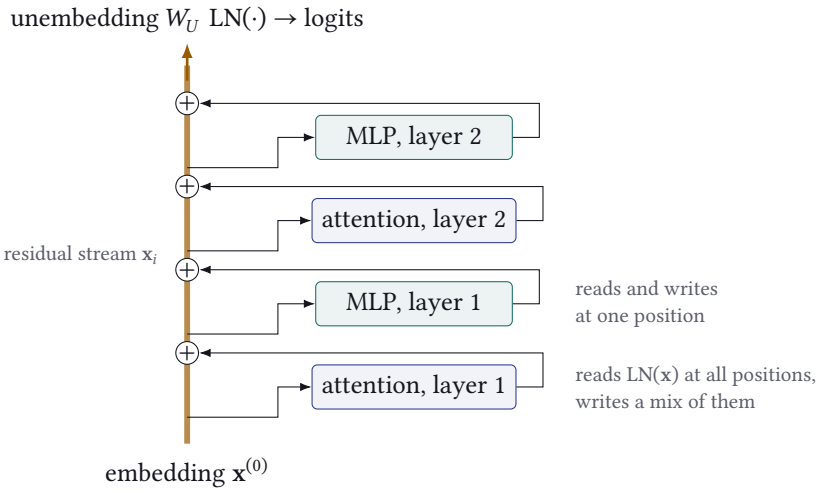


Figure 7. The residual-stream view of a two-layer transformer at one position. Every component reads the stream (through a layer norm) and adds its output back; the final stream is the embedding plus the sum of all outputs.

Heads as two circuits. Ignoring the layer norm, an attention head r writes $\sum_j A_{ij}^{(r)} U^{(r)} \mathbf{x}_j$ to position i, where $W_O^{(r)} \in \mathbb{R}^{d_k \times d}$ is the block of rows of W_O that multiplies the output of head r (so $[H_1, \dots, H_h] W_O = \sum_r H_r W_O^{(r)}$), $U^{(r)} = (W_V^{(r)} W_O^{(r)})^\mathsf{T} \in \mathbb{R}^{d \times d}$, and $A_{ij}^{(r)} \propto \exp(\mathbf{x}_i^\mathsf{T} B^{(r)} \mathbf{x}_j / \sqrt{d_k})$, $B^{(r)} = W_Q^{(r)} W_K^{(r)\mathsf{T}}$. Two low-rank matrices thus describe a head: the *QK circuit* $B^{(r)}$ decides *where* to read, and the *OV circuit* $U^{(r)}$ decides *what* is written once the position is chosen [13]. A head can attend strongly to a token and write almost nothing, if $U^{(r)}$ is small in the relevant directions.

Direct logit attribution. Freeze the scale factor of the final layer norm at its value on a given input. Then the logits in (20) are a *sum* of contributions, one per component, plus the embedding. For two tokens a and b the *logit difference* splits as

$$\text{logit}_a - \text{logit}_b \approx \sum_c \frac{(\mathbf{u}_a - \mathbf{u}_b)^\top \mathbf{o}^{(c)}}{\sigma} + (\text{embedding term}),$$

with $\mathbf{u}_a, \mathbf{u}_b$ rows of W_U (absorbing the layer-norm gain) and σ the frozen scale. Each term measures how much a component pushes the prediction *directly*. It misses indirect effects: a component whose output is read by a later component, rather than by the unembedding, can be essential and still have a direct contribution of zero.

7.2 What attention patterns show, and what they do not

The weights $A_{ij}^{(r)}$ of every head can be drawn as a $T \times T$ image, and some heads have very recognisable patterns: attend to the previous token, to the first token, to the same token earlier in the context. A well-studied example is the *induction head* [13, 32]. In a context of the form "… A B … A", it attends from the second A to the token B that followed the first A, and copies B, predicting that the pattern repeats. It needs a second head in an earlier layer that writes "the previous token was A" into the stream at the position of B: two heads composed through the residual stream. Olsson et al. found that induction heads appear in a sudden change during training, at the same time as a marked improvement of the model's ability to use earlier parts of its context.

Patterns are evidence, but they are not explanations by themselves.

- A pattern shows where a head *reads*, not what it *writes*: the OV circuit may send almost nothing, or something no later component uses.
- In a deep model, position j of layer l no longer contains only token j; after a few layers it mixes information from many positions, so "attends to token j" is ambiguous.
- For recurrent models with attention, Jain and Wallace found that the weights often correlate poorly with other measures of importance and that quite different weight distributions can give the same prediction [25]. Wiegreffe and Pinter showed that the conclusion depends on how the test is set up [45] — which is the point: the weights alone do not settle the question.

To learn what a component *does*, one must intervene on it.

7.3 Inspection and ablation

Inspection records activations: every head's pattern and output, every MLP's hidden units, the stream after every layer, for a set of inputs. Frameworks register "hooks" on modules that store or modify their outputs during the forward pass. Inspection finds correlations; *ablation* tests causation by removing a component and measuring the effect on a metric such as the logit difference or the loss. *Zero ablation* sets the component's output to $\mathbf{0}$; simple, but a vector of zeros may never occur in normal operation, and the response to such an off-distribution input can exaggerate or invent effects. *Mean ablation* replaces the output by its mean over a reference set of inputs, so the component keeps its average contribution and loses only its dependence on the current input; *resample ablation* uses its value on another, random input. A large drop of the metric shows that the component is needed for the behaviour; a small drop does not prove the opposite, because other components may compensate (redundancy, sometimes called self-repair). Algorithm 9 below covers ablation and patching in one procedure.

7.4 Activation patching

Activation patching asks a sharper question with two inputs that differ in one controlled respect: a *clean* input on which the model behaves as intended, and a *corrupted* input that changes the answer. For example, "The Eiffel Tower is in the city of" (answer: Paris) and "The Colosseum is in the city of" (answer: Rome). The two inputs are chosen to have the same number of tokens, so that position i means the same thing in both runs. Run both, record the activations of the clean run, then rerun the corrupted input while replacing *one* activation by its clean value. If the output moves back towards the clean answer, that activation carries the information that distinguishes the two inputs. With the metric m (a logit difference) the *restored fraction* is

$$\rho(c) = \frac{m_{\text{patched}}(c) - m_{\text{corrupt}}}{m_{\text{clean}} - m_{\text{corrupt}}}, \tag{21}$$

1 for full restoration and 0 for none. Patching clean into corrupted (*denoising*) tests whether a component is *sufficient* to restore the behaviour; patching corrupted into clean (*noising*) tests whether it is *necessary*; the two can disagree [23]. Meng et al. used this kind of experiment, with noise added to the embeddings of the subject as the corruption, to locate factual recall in GPT-type models in middle-layer MLPs at the last token of the subject [30]; Wang et al. used patching of paths between components to identify a circuit of attention heads in GPT-2 small that completes sentences such as "When Mary and John went to the store, John gave a drink to" with "Mary" [44].

Worked example — 10. Patching a model small enough to compute by hand

A model sees two tokens, each $+1$ or -1, and must report the first one at the last position through the score $y = \text{logit}_+ - \text{logit}_-$. Its residual stream at the last position has three dimensions, and the unembedding reads y from the second one, $\mathbf{u} = (0, 1, 0)$. Its components (layer norms omitted) are

- the embedding of the second token, $(0,\ 0.5\,x_2,\ 0)$;
- two heads in layer 1 that attend to the first token, H1 writing $(0.75\,x_1, 0, 0)$ and H2 writing $(0.25\,x_1, 0, 0)$;
- an MLP in layer 1, M, that reads dimension 1 of the stream after the heads and writes its sign into dimension 3, $(0, 0, \text{sign}(\cdot))$, so $(0, 0, x_1)$ in both unpatched runs;
- a component R in layer 2 that reads dimension 1 of the stream and writes twice its value into dimension 2.

Clean input $(x_1, x_2) = (+1, +1)$: before R the stream is $(1, 0.5, 1)$, R adds $(0, 2, 0)$, and $y = 2.5$. *Corrupted* input $(-1, +1)$: the stream is $(-1, 0.5, -1)$, R adds $(0, -2, 0)$, and $y = -1.5$. The difference is 4.

patched into the corrupted run	dimension 1 before R	y	restored fraction ρ
nothing	-1	-1.5	0
H1 (clean output 0.75)	$0.75 - 0.25 = 0.5$	1.5	0.75
H2 (clean output 0.25)	$-0.75 + 0.25 = -0.5$	-0.5	0.25
H1 and H2	1	2.5	1
M	-1	-1.5	0
R (clean output 2)	-1	2.5	1
embedding (identical in both runs)	-1	-1.5	0

The experiment recovers the mechanism: the information flows from the first token through H1 and H2, which share the work three to one, into R, which writes the answer. Zero ablation in the clean run agrees on necessity (y drops from 2.5 to 1.0 without H1, to 2.0 without H2,

and stays at 2.5 without M). Two lessons carry over to real models. If H1 and H2 both attend fully to the first token, their patterns are identical, yet their effects differ by a factor of three. And a linear probe on dimension 3 decodes x_1 perfectly from the output of M, yet patching or ablating M changes nothing: the information is present but unused.

Algorithm 9 Activation patching and ablation over a set of components

Require: model; clean input $\mathbf{x}^{\text{clean}}$ and corrupted input $\mathbf{x}^{\text{corr}}$; metric m; components $c = 1, \dots, C$ and positions $i = 1, \dots, P$; base input (corrupted for denoising, clean for noising and ablation); replacement $r(c, i)$: clean value $a_{c,i}^{\text{clean}}$, corrupted value, $\mathbf{0}$, mean or resampled value

function PATCH($\mathbf{x}^{\text{clean}}, \mathbf{x}^{\text{corr}}$)
  run on $\mathbf{x}^{\text{clean}}$, record all activations $a_{c,i}^{\text{clean}}$ and m_{clean} ▷ hooks store outputs
  run on $\mathbf{x}^{\text{corr}}$, record m_{corrupt} and the activations
  for each component c and position i **do**
    rerun the base input with the output of c at i set to $r(c, i)$ ▷ rest recomputed
    if patching **then**
      $\rho(c, i) \leftarrow (m_{\text{patched}} - m_{\text{corrupt}})/(m_{\text{clean}} - m_{\text{corrupt}})$
    else
      $\Delta(c, i) \leftarrow m_{\text{base}} - m_{\text{ablated}}$ ▷ drop of the metric
    end if
  end for
  return the table $\rho(c, i)$ or $\Delta(c, i)$ ▷ average over many clean/corrupted pairs
end function

Running time — activation patching

Time: two reference runs plus one forward pass per patched (component, position) pair, $O(CPF)$ for a forward cost F; the part of each pass below the patched layer can be reused from the cached corrupted run. For all heads of a 12-layer, 12-head model at 20 positions this is 2880 forward passes per pair of inputs; a first-order approximation (attribution patching) estimates all $\rho(c, i)$ from one backward pass [23] and helps decide where to patch exactly. The cost is the same for all replacement rules; mean ablation needs one extra pass over the reference set. *Memory*: the cached activations of the clean and the corrupted run, $O(LPd)$ plus the attention patterns if heads are patched. Results for one pair are anecdotes: average over many pairs, and choose the corruption with care, since it defines the question asked.

7.5 Probes and causal tests

A *probe* is a simple classifier — usually logistic regression (Note 9) — trained to predict a property (the part of speech of a word, the parity of a count, the colour of a square on a board) from the activations at some layer. High probe accuracy shows that the property is *decodable* from the activations. It does not show that the model *uses* it, as Worked example 10 illustrates: the decodable information may be a by-product that no later component reads.

- An expressive probe can learn the task itself rather than read it off. Hewitt and Liang compare each probe with a *control task* that assigns random but consistent labels to the same inputs; a good probe should be accurate on the real property and poor on the control, i.e. *selective* [24]. Linear probes are preferred for this reason.
- Correlation is not causation. A probe finds a direction that correlates with the property; whether the model's output depends on that direction must be tested by intervening along it — ablating it or patching it — as in Section 7.4. Belinkov's survey discusses these limitations

and remedies [5].

In short: inspection and probes generate hypotheses, ablation and patching test them, and a claim about a mechanism should survive both.

Summary Attention replaces the fixed vector of an encoder–decoder by a softmax-weighted average of all encoder states, recomputed at every output step; the gradient of a score is its weight times the excess of its value's loss gradient g_m over the average $\bar{g}$. Location-based windows and CTC handle monotonic alignments, the latter by exact dynamic programming over all paths. Hard attention samples one location, costs $O(1)$ in the input size and is trained with the REINFORCE estimator, whose variance a baseline reduces without bias. Self-attention computes $\mathrm{softmax}(QK^{\mathsf{T}}/\sqrt{d_k} + M)V$ with learned projections, the scale keeping the scores at unit variance; it is permutation equivariant, so positional information is supplied by added sinusoids or learned vectors, or by rotating queries and keys. Multi-head attention and position-wise MLPs with residual connections form the transformer layer, at cost $12Td^2 + 2T^2d$. Decoder-only models generate with a key–value cache whose size $2LTd$ limits long contexts; tiled attention computes exact attention without storing the $T \times T$ matrix. Inside a transformer, components read from and write to a residual stream, and causal interventions, not patterns or probes alone, establish what they do.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. State the two weaknesses of an encoder–decoder that passes a single fixed vector from the encoder to the decoder. Explain how soft attention removes each of them.

A2. In Figure 2, the Turkish word "kitabı" reads mostly the English "book" and "the". Why would a forward-only encoder state at "the" be a poor key for it, and why does a bidirectional encoder help? Which rows of the figure show reordering, and which show one output word doing the work of two input words?

A3. Interpret the context vector $\mathbf{c}_i$ as an expectation. Using (6), explain why an input position with $\alpha_{ij} \approx 0$ receives almost no gradient, and why a saturated (nearly one-hot) attention distribution stops learning.

A4. How does the Gaussian window of Graves guarantee a monotonic alignment? For which tasks is this desirable, and what is the price?

A5. What is the role of the blank symbol in CTC? Why must a blank separate two equal letters, what independence assumption does (9) make, and why can its sum over exponentially many paths be computed in time $O(T|\mathbf{l}|)$?

A6. List four differences between soft and hard attention. Why can hard attention not be trained by backpropagation alone, and why is a glimpse model's cost independent of the image size?

A7. Explain how DRAW reads a patch of an image through a differentiable window. Why can

it be trained by backpropagation while the recurrent attention model cannot? Would such a window suit speech recognition, given the shape of its alignments?

A8. Why are the scores divided by $\sqrt{d_k}$? Describe what happens to the attention weights and their gradients without this factor when d_k is large.

A9. What does it mean that self-attention is permutation equivariant? Why does a language model need positional information, while a model for the particles of a jet may not?

A10. What does a causal mask do, and why does it allow a decoder to be trained on all positions of a sequence in one pass? What is a padding mask for?

A11. Why do the number of parameters and the number of multiply–adds of multi-head attention not depend on the number of heads h when $d_k = d/h$? What does grow with h? What do several heads offer that one head of width d does not?

A12. For each of the three kinds of attention in the encoder–decoder transformer, say where the queries, keys and values come from and which mask is used.

A13. Compare sinusoidal encodings, learned position embeddings and rotary embeddings. Which of them makes the score depend on relative positions only?

A14. Compare a recurrent layer and a self-attention layer for a sequence of length T: cost, number of sequential steps, and length of the path between two positions. For which T do the attention scores and weighted sums cost as much as all the matrix products (projections and MLP with $d_{\text{ff}} = 4d$) of a transformer layer?

A15. What exactly does a key–value cache store, and why may stored keys and values be reused at later steps? Why is token-by-token generation limited by memory bandwidth, and how do multi-query and grouped-query attention help?

A16. FlashAttention performs at least as many arithmetic operations as standard attention, yet it is faster and its result is exact. Explain both facts, and say what it stores instead of the $T \times T$ weight matrix.

A17. Distinguish zero ablation, mean ablation and activation patching, and say what a restored fraction of 1 means.

A18. Explain the residual-stream view of a transformer. Why does it make direct logit attribution possible, and what does direct logit attribution miss?

A19. Give two reasons why an attention pattern alone does not explain what a head contributes.

A20. A linear probe recovers a property from layer 5 with 98% accuracy. Does the model use the property? Describe a control and a causal test you would run.

B. Short calculations

B1. Let $\mathbf{h}_1 = (2, 0)$, $\mathbf{h}_2 = (0, 1)$, $\mathbf{h}_3 = (1, 1)$ and $\mathbf{s} = (0.5, 1)$. Compute the dot-product scores, the attention weights and the context vector. Repeat with the scaled dot product ($d = 2$). Which weights change most?

B2. For the dot-product weights of Question B1 and the loss $\ell = \mathbf{u}^{\mathsf{T}}\mathbf{c}$ with $\mathbf{u} = (0, 1)$, compute g_j, $\bar{g}$ and $\partial\ell/\partial e_j$ for $j = 1, 2, 3$ using (6). Check that the three numbers sum to zero, and give $\partial\ell/\partial\mathbf{h}_2$

through the weighted sum.

B3. A Graves window has $K = 1$, $\alpha = 1$, $\beta = 1$, $\kappa_0 = 0$ and offsets $\exp(\hat{\kappa}_t) = 1.3$ and 0.5. Compute $\phi(t, u)$ for $t = 1, 2$ and $u = 1, 2, 3, 4$, and say which character dominates the window at each step.

B4. CTC with alphabet $\{\mathtt{a}, \mathtt{b}\}$, $T = 3$ frames and outputs $y^1 = (0.5, 0.2, 0.3)$, $y^2 = (0.4, 0.1, 0.5)$, $y^3 = (0.2, 0.2, 0.6)$ for $(\mathtt{a}, \mathtt{b}, -)$. (a) List all paths that collapse to $\mathtt{a}$ and compute $p(\mathtt{a} \mid \mathbf{x})$. (b) Compute the same number with the recursion (10) on $\mathbf{l}' = (-, \mathtt{a}, -)$, writing out the 3×3 table.

B5. A policy chooses location 1 with probability $\pi = \sigma(\theta)$ and location 2 otherwise; the rewards are $R = 3$ and $R = 1$. At $\theta = \ln 3$, compute π, the exact gradient $\partial \mathbb{E}[R]/\partial\theta$, the two possible single-sample REINFORCE estimates and their variance, first without a baseline and then with $b = \mathbb{E}[R]$.

B6. Three tokens have $\mathbf{x}_1 = (1, 1)$, $\mathbf{x}_2 = (1, 0)$, $\mathbf{x}_3 = (0, 2)$, and $W_Q = W_K = W_V = I$, $d_k = 2$. (a) Compute $QK^\mathsf{T}/\sqrt{2}$ and the attention weights of token 3. (b) Apply a causal mask and compute the outputs of tokens 1 and 2. (c) Swap tokens 1 and 2 without a mask and verify that the output of token 3 does not change. (d) Compute the unmasked output of token 3 again, processing its keys in the two blocks $\{1, 2\}$ and $\{3\}$ with the online softmax (19), and check that it agrees with the direct computation.

B7. A decoder-only model has $d = 1024$, $h = 16$, $d_{\text{ff}} = 4096$, $L = 24$ layers and a vocabulary of $V = 32\,000$ tokens with tied embedding and unembedding. (a) Count the parameters of one layer (attention, MLP, two layer norms, all with biases) and of the whole model, including the final layer norm but ignoring position embeddings. (b) Compute the KV cache per token and for 2048 tokens in 16-bit numbers. (c) For which sequence length do the attention scores cost as much as the rest of a layer?

B8. In the model of Worked example 10, change H1 to write $(0.4\,x_1, 0, 0)$, H2 to write $(0.6\,x_1, 0, 0)$ and let M write $(0.5\,s, 0, s)$ instead of $(0, 0, s)$, where s is the sign of dimension 1 after the heads, recomputed whenever a head is patched. Recompute the clean and corrupted scores and the restored fractions for patching H1, H2 and M. What does a probe on dimension 3 now tell you?

C. Derivations

C1. Derive the Jacobian of the softmax and (6). Then, for dot-product attention with scores $e_j = \mathbf{q}^\mathsf{T}\mathbf{k}_j/\sqrt{d_k}$, express $\partial\ell/\partial\mathbf{q}$ and $\partial\ell/\partial\mathbf{k}_j$ through the $\partial\ell/\partial e_j$. Finally, for the matrix form $O = \text{softmax}(S)V$ with $S = QK^\mathsf{T}/\sqrt{d_k}$, show that $\partial\ell/\partial V = A^\mathsf{T}\,\partial\ell/\partial O$, $\partial\ell/\partial A = (\partial\ell/\partial O)V^\mathsf{T}$, and give $\partial\ell/\partial S$ row by row.

C2. Show that $\text{Var}(\mathbf{q}^\mathsf{T}\mathbf{k}) = d_k\sigma_q^2\sigma_k^2$ when all components are independent with mean zero and variances σ_q^2 and σ_k^2. Assuming the scores of n keys are independent $\mathcal{N}(0, s^2)$, argue why the largest attention weight approaches 1 as s grows, and connect this with the $\tau \to 0$ limit of the physics box of Section 2.5.

C3. Prove that unmasked self-attention is permutation equivariant and that a causal mask breaks this property. Prove that the sinusoidal encodings satisfy $\mathbf{p}_{t+k} = R_k\mathbf{p}_t$ with R_k orthogonal and independent of t, and that with rotary embeddings the score $(R_m\mathbf{q})^\mathsf{T}(R_n\mathbf{k})$ depends on m and n only through $n - m$.

C4. Derive the REINFORCE identity (12) for a continuous distribution, show that a constant base-

line leaves it unbiased, and find the constant b^* that minimises the variance of the estimator $(R - b)\,\partial_\theta \log p_\theta(z)$ for a scalar parameter θ. Evaluate b^* for the policy of Worked example 5 at $\theta = 1$ with the rewards $R = 1$ (left) and $R = 0$ (right), and again with $R = 11$ and $R = 10$; compare b^* with $\mathbb{E}[R]$.

D. Programming

D1. In numpy, implement Algorithm 5 for a batch of sequences with causal and padding masks, and its backward pass using the results of Question C1. Check the gradients by central differences (Note 12, Section 4). Compare the forward pass with PyTorch's `scaled_dot_product_attention` (in `torch.nn.functional`). Verify numerically that the unmasked layer is permutation equivariant.

D2. Implement Algorithm 8 in numpy with blocks of B_k keys and check that it agrees with the direct formula to round-off for random inputs and several B_k, with and without a causal mask. Then write a tiny decoder-only model with random weights and implement generation with and without a key–value cache (Algorithm 7); check that both give the same greedy tokens, and time them for $N = 50, 100, 200$.

D3. Train a two-layer, attention-only decoder (no MLPs) on sequences made of a random segment of 20 tokens repeated twice. Plot the attention patterns of all heads on a test sequence and identify a previous-token head and an induction head. Measure the loss on the second half after zero and mean ablation of each head. Then run activation patching (Algorithm 9) between a clean sequence and a corrupted one in which the first copy of the segment is replaced by different tokens, and train a linear probe for "the previous token" on the stream after layer 1. Do the methods agree?

Further reading. Goodfellow, Bengio and Courville [16], Section 12.4.5, for attention in neural machine translation; Prince [33], chapter 12, and Bishop and Bishop [6], chapter 12, for transformers. For interpretability, start with the "Reading list: mechanistic interpretability" on the course page, then Elhage et al. [13] on the mathematical framework of the residual stream and of QK and OV circuits, Olsson et al. [32] on induction heads, Meng et al. [30] on locating factual recall, Wang et al. [44] on a circuit in GPT-2 small, Heimersheim and Nanda [23] on the practice of patching, and Hewitt and Liang [24] and Belinkov [5] on probes.

References

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón and S. Sanghai, GQA: training generalized multi-query transformer models from multi-head checkpoints, *EMNLP* (2023).

[2] J. Ba, V. Mnih and K. Kavukcuoglu, Multiple object recognition with visual attention, *ICLR* (2015).

[3] A. Baevski, Y. Zhou, A. Mohamed and M. Auli, wav2vec 2.0: a framework for self-supervised learning of speech representations, *NeurIPS* (2020).

[4] D. Bahdanau, K. Cho and Y. Bengio, Neural machine translation by jointly learning to align and translate, *ICLR* (2015).

[5] Y. Belinkov, Probing classifiers: promises, shortcomings, and advances, *Comput. Linguist.* 48, 207–219 (2022).

[6] C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer (2024).

[7] T. B. Brown *et al.*, Language models are few-shot learners, *NeurIPS* (2020).

[8] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk and Y. Bengio, Learning phrase representations using RNN encoder–decoder for statistical machine translation, *EMNLP* (2014).

[9] J. Chorowski, D. Bahdanau, D. Serdyuk, K. Cho and Y. Bengio, Attention-based models for speech recognition, *NeurIPS* (2015).

[10] T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, FlashAttention: fast and memory-efficient exact attention with IO-awareness, *NeurIPS* (2022).

[11] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, *NAACL* (2019).

[12] A. Dosovitskiy *et al.*, An image is worth 16×16 words: transformers for image recognition at scale, *ICLR* (2021).

[13] N. Elhage *et al.*, A mathematical framework for transformer circuits, *Transformer Circuits Thread* (2021).

[14] W. Fedus, B. Zoph and N. Shazeer, Switch transformers: scaling to trillion parameter models with simple and efficient sparsity, *J. Mach. Learn. Res.* 23, 1–39 (2022).

[15] B. Geshkovski, C. Letrouit, Y. Polyanskiy and P. Rigollet, A mathematical perspective on transformers, arXiv:2312.10794 (2023).

[16] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press (2016).

[17] A. Graves, S. Fernández, F. Gomez and J. Schmidhuber, Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks, *ICML* (2006).

[18] A. Graves, Sequence transduction with recurrent neural networks, arXiv:1211.3711 (2012).

[19] A. Graves, Generating sequences with recurrent neural networks, arXiv:1308.0850 (2013).

[20] K. Gregor, I. Danihelka, A. Graves, D. J. Rezende and D. Wierstra, DRAW: a recurrent neural network for image generation, *ICML* (2015).

[21] A. Gu, K. Goel and C. Ré, Efficiently modeling long sequences with structured state spaces, *ICLR* (2022).

[22] A. Gu and T. Dao, Mamba: linear-time sequence modeling with selective state spaces, arXiv:2312.00752 (2023).

[23] S. Heimersheim and N. Nanda, How to use and interpret activation patching, arXiv:2404.15255 (2024).

[24] J. Hewitt and P. Liang, Designing and interpreting probes with control tasks, *EMNLP* (2019).

[25] S. Jain and B. C. Wallace, Attention is not explanation, *NAACL* (2019).

[26] J. Jumper *et al.*, Highly accurate protein structure prediction with AlphaFold, *Nature* 596, 583–589 (2021).

[27] N. Kalchbrenner and P. Blunsom, Recurrent continuous translation models, *EMNLP* (2013).

[28] M.-T. Luong, H. Pham and C. D. Manning, Effective approaches to attention-based neural machine translation, *EMNLP* (2015).

[29] Y. Luo, C.-C. Chiu, N. Jaitly and I. Sutskever, Learning online alignments with continuous rewards policy gradient, arXiv:1608.01281 (2016).

[30] K. Meng, D. Bau, A. Andonian and Y. Belinkov, Locating and editing factual associations in GPT, *NeurIPS* (2022).

[31] V. Mnih, N. Heess, A. Graves and K. Kavukcuoglu, Recurrent models of visual attention, *NeurIPS* (2014).

[32] C. Olsson *et al.*, In-context learning and induction heads, *Transformer Circuits Thread* (2022).

[33] S. J. D. Prince, *Understanding Deep Learning*, MIT Press (2023).

[34] H. Qu, C. Li and S. Qian, Particle transformer for jet tagging, *ICML* (2022).

[35] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever, Language models are unsupervised multitask learners, OpenAI technical report (2019).

[36] A. Radford *et al.*, Learning transferable visual models from natural language supervision, *ICML* (2021).

[37] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, Robust speech recognition via large-scale weak supervision, *ICML* (2023).

[38] H. Ramsauer *et al.*, Hopfield networks is all you need, *ICLR* (2021).

[39] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton and J. Dean, Outrageously large neural networks: the sparsely-gated mixture-of-experts layer, *ICLR* (2017).

[40] N. Shazeer, Fast transformer decoding: one write-head is all you need, arXiv:1911.02150 (2019).

[41] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo and Y. Liu, RoFormer: enhanced transformer with rotary position embedding, *Neurocomputing* 568, 127063 (2024).

[42] I. Sutskever, O. Vinyals and Q. V. Le, Sequence to sequence learning with neural networks, *NeurIPS* (2014).

[43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, Attention is all you need, *NeurIPS* (2017).

[44] K. Wang, A. Variengien, A. Conmy, B. Shlegeris and J. Steinhardt, Interpretability in the wild: a circuit for indirect object identification in GPT-2 small, *ICLR* (2023).

[45] S. Wiegreffe and Y. Pinter, Attention is not not explanation, *EMNLP-IJCNLP* (2019).

[46] R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, *Mach. Learn.* 8, 229–256 (1992).

[47] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel and Y. Bengio, Show, attend and tell: neural image caption generation with visual attention, *ICML* (2015).

The selection and order of topics in this note follow the lecture on attention models of the course IFT 6135 *Representation Learning* at Université de Montréal and Mila (slides by Dzmitry Bahdanau, edited by Aaron Courville). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures_note15.py`, published with the sources of these notes.